草履虫都能学会的Git与GitHub使用指南

写在前面

本教程由 **北京林业大学森林狼机器人战队 Lee** 编写。初衷是为了战队内部的交流与培训，旨在进行"计算机扫盲"，帮助大家切实提升工具使用水平。

教程内容涵盖三种 Git 使用方式，同学们可以根据自身情况和学习阶段灵活选择：

1. **命令行模式（推荐掌握）**：本教程的核心内容，使用原生命令行操作。这是最通用、最强大的方式，能让你深入理解 Git 的工作原理，适用于所有操作系统和复杂场景。
2. **图形化界面（GUI）模式**：通过可视化界面进行 Git 操作，直观易懂，适合初学者快速上手。常见工具包括 Sourcetree、GitKraken、VSCode 内置 Git 等。本教程将在相关章节提供 GUI 操作对照说明。
3. **AI 辅助模式**：借助 AI 工具（如 GitHub Copilot、通义灵码、Cursor 等）自动生成 Git 命令、解释错误信息、提供操作建议。这是面向未来的开发方式，能显著提升效率，但需要你对 Git 有基本理解才能正确使用 AI 的建议。

学习建议：**新手建议从 GUI 入门建立信心，同时逐步学习命令行掌握核心技能，熟练后借助 AI 提升效率**。三线并行，循序渐进，让 Git 真正成为你的得力助手。

*注：本文仅作**内部读物**分享，主打通俗易懂，水平有限，大佬莫笑！*

前言：为什么要使用GitHub与Git

简单说，就是为了让你**写代码更从容，抄代码更安全**：

- **自带“后悔药”**：改崩了？误删了？一键回退到昨天的版本，彻底告别 `最终版_v2_改_再改_绝对不改了.zip`。
- **拒绝“二道贩子”**：直接从原作者手里拿代码，没有广告弹窗，没有中间商加料篡改，原汁原味。
- **队友“不打架”**：你写车轮，我写底盘，自动合并在一起。不用拿U盘拷来拷去，更不会不小心覆盖掉队友通宵写的代码。
- **团队“藏经阁”**：文档、代码、资料都在云端，新人进队一目了然，不用追着老学长要文件。

草履虫都能学会的Git与GitHub使用指南
前言：为什么要使用GitHub与Git
准备工作
一、GitHub账号注册与登录
二、软件下载安装与配置入门
下载并安装 Git（必装软件）
下载并安装 GitHub Desktop（图形化软件，新手推荐）
安装并配置 AI 编程助手
使用教程
一、图形化界面（GUI）快速上手（新手推荐）
1.1 GitHub Desktop 首页认识
1.2 创建/克隆仓库

1.2.1 创建云端仓库
1.2.2 创建本地仓库并发布到云端
1.2.3 克隆他人或云端仓库（最常用）
1.3 查看文件变更
1.4 提交代码
1.5 推送到远程
1.6 其他操作后续补充
1.7 常用 GUI 工具对比
二、AI 辅助提效（现代开发方式，后续补充）
2.1 通义灵码使用指南
2.2 其他 AI 工具
三、命令行基础教程（必须掌握）
3.1 Git 基础概念（先了解）
3.2 创建仓库（从零开始）
3.2.1 本地无文件，从 GitHub 创建全新仓库
3.2.2 本地有文件，绑定到 GitHub
3.2.3 下载他人现成的仓库
3.3 日常开发核心指令（高频使用）
3.3.1 三板斧：Add → Commit → Push
3.3.2 查看状态与历史
3.3.3 版本管理
3.3.4 分支操作
3.4 常见问题自救指南
3.5 常用指令速查表
1、如何创建仓库（基础）
1、自己创建一个新的仓库
第一步：在 GitHub 建立新仓库
第二步：在本地创建仓库并与GitHub绑定
情况 A：本地没有文件，创建全新仓库
情况 B：本地有文件，将本地仓库与GitHub仓库连接
2、从 GitHub 下载他人仓库（`git clone`）
补充教程：初始化本地仓库
四、常用基础指令（日常开发）
1 、三条核心指令：Add, Commit, Push
🚑 紧急救援：第一次 Push 报错了？
2、查看历史记录(`git log`)
3 版本回退（`git reset`）
⚠️ 安全须知
4、检查当前状态（`git status`）
5、拉取最新代码（`git pull`）
6、给分支改名 (git branch -M)
待补充更新！
5、进阶高级指令（多人协作）
待补充更新！
常用指令速查表
常见问题解决办法
1. 推送报错，云端与本地文件不同步，`! [rejected] ... (fetch first)`

准备工作

一、GitHub账号注册与登录

在这个阶段，我们将获得进入代码世界的“身份证”。请务必一步步跟着做，不要跳过。

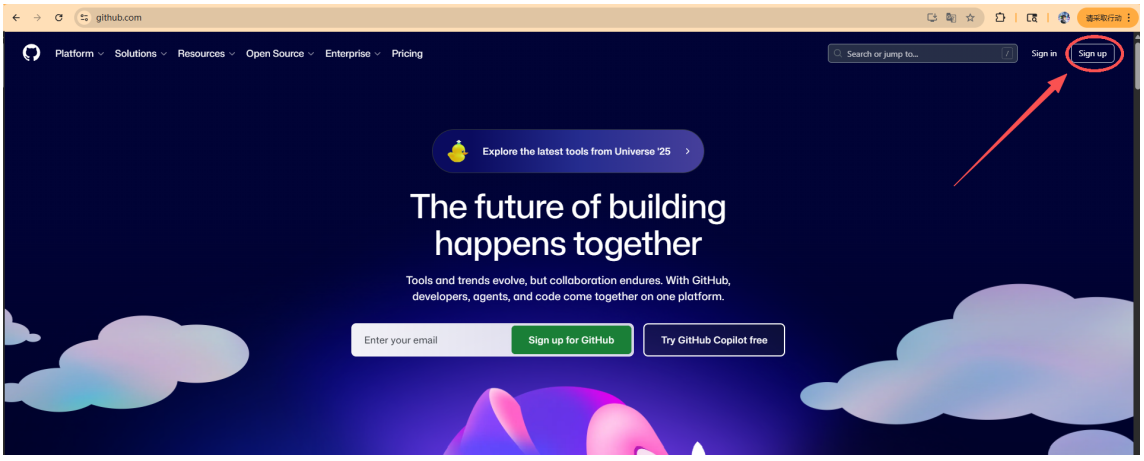
第一步：准备工作

在打开网页之前，你需要准备好以下三样东西：

1. **稳定的邮箱**：推荐用 Gmail、Outlook，国内的 QQ 邮箱、163 邮箱也可以（由于 GitHub 服务器在海外，QQ 邮箱偶尔可能会收不到邮件，如果在垃圾箱也没翻到，就换个邮箱）。
2. **网络环境（科学上网）**：**【重点】** 虽然 GitHub 在国内偶尔能直接访问，但极其不稳定。为了后续不报错，**强烈建议开启“科学上网”工具。**

第二步：进入官网注册

1. 打开浏览器（推荐 Chrome 或 Edge），输入官网地址：`https://github.com`
2. 在首页右上角，点击白色的 **“Sign up”**（注册）按钮。

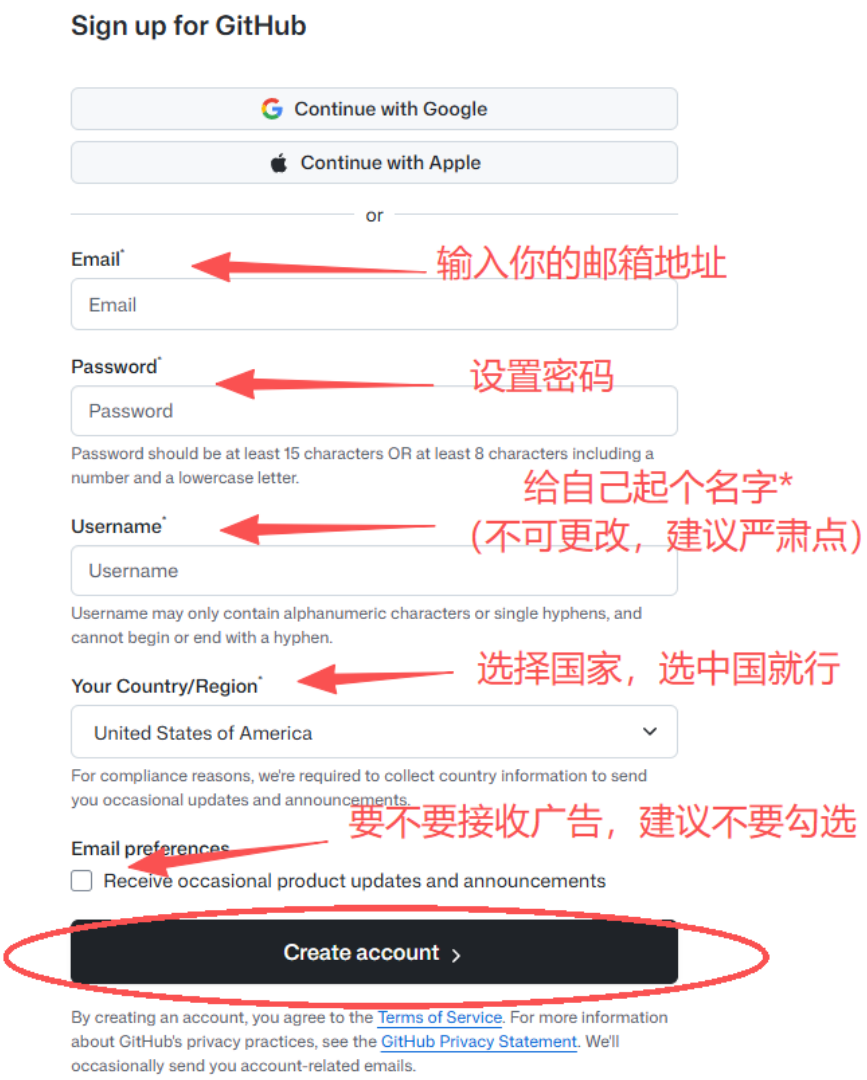


第三步：填写信息

GitHub现在的注册页面会一行一行问你：

1. **email**：输入你的邮箱地址
2. **Create a password**：设置密码。
   - *注意：密码需要足够强（包含大小写字母+数字/符号），如果你输入的密码显示红字，说明太简单了，加点符号试试。*
3. **Enter a username**：给自己起个名字。
   - *警告：* 这个名字会显示在你的主页地址栏里（例如 `github.com/你的名字`），也会出现在你未来的代码提交记录里。**建议起个正经点的英文名（如 `Sanzhang2024`），千万别乱敲脸滚键盘，因为这可能会出现在你未来的求职简历上！**
4. **Email preferences**：问你要不要收广告邮件。不要勾选，然后点 `Continue`。

5.

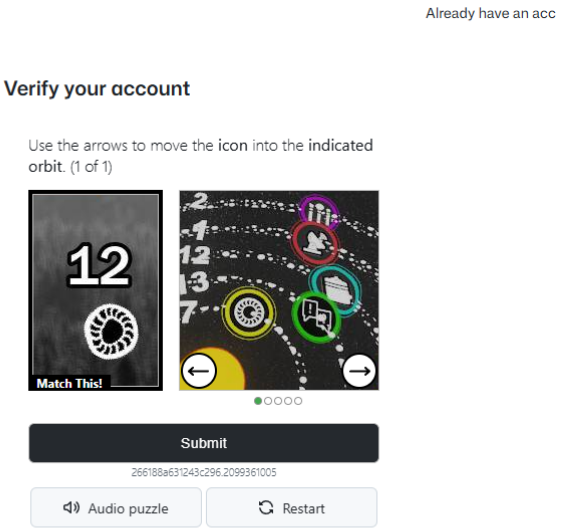


第四步：真人验证

接下来会出现一个**验证码环节**（Verify your account）。

- 通常是“选出两个一样的图标”或者“把动物旋转到正确方向”或者“计算螺旋星系”。
- *吐槽：* 有时候这个验证很变态，如果失败了多试几次，耐心一点。

验证通过后，点击最下方的 **“Create account”** 按钮。



第五步：邮箱验证码

GitHub 会向你的邮箱发送一串数字验证码。

- 去你的邮箱找到邮件。
- 去你的邮箱收信（如果没有，翻翻垃圾箱，或者看看是不是邮箱地址填错了）。
- 邮件标题通常叫 *Your GitHub launch code*，里面有一串显眼的 **8位数字。**
- 把数字填回到网页里。

第六步：登录账号

完成上面验证后你会回到主页

- 在首页右上角，点击白色的 **“Sign in”**（登录）按钮。
- 输入你刚才注册的账号密码
- 通过你的邮箱接受验证码
- 完成登录

第七步：大功告成

完成上述步骤后，当你看到一个左上角有八爪鱼猫头像（GitHub图标），中间可能写着 "Dashboard" 或者让你 "Create a new repository" 的页面时，**恭喜你！注册流程彻底结束了。**

给草履虫的总结：

1. 开梯子。
2. 起个好听的英文名（别乱填）。

重要补充：GitHub 双重验证 (2FA) 预警

GitHub 会不定期强制用户开启 **2FA（双重身份验证）**。若你在登录时被拦截，请参考以下说明：

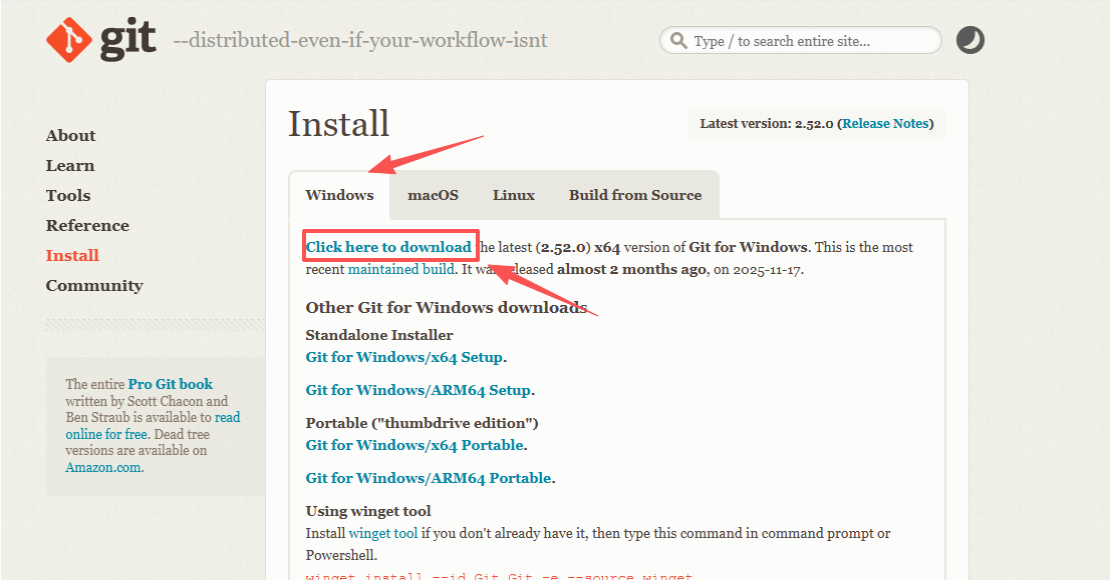
- **验证机制**：开启后，登录不仅需要密码，还需输入手机 App 实时生成的 6 位动态验证码。
- **触发标志**：若页面弹出 **Setup Two-factor authentication**，即表示系统已强制要求你完成配置。
- **解决方案**：此时请直接跳转至本手册附录的 **《2FA 开启与恢复指南》**。

二、软件下载安装与配置入门

下载并安装 Git（必装软件）

1. 下载软件

1. 访问 Git 官方下载页面：git-scm.com/download/win
2. 点击 **"Click here to download"** 下载最新的 64 位安装程序。



2. 安装过程中的关键设置

1. 双击运行下载好的 `.exe` 文件。
2. **核心策略**：安装程序会问你一大堆问题，如果你看不懂，**请一路狂点 "Next"（下一步），直到 "Install"（安装）**。默认设置已经够用了！
3. *进阶一点点（仅供学有余力的草履虫参考）：*
   - 选择组件 (**Select Components**)：建议勾选 Git Bash Here 和 Git GUI Here，这样你可以在任何文件夹右键快速打开 Git。
   - 选择默认编辑器**(Choosing the default editor)**：如果你平时用 VS Code，可以在下拉菜单选 Visual Studio Code；如果没有偏好，就选默认的 Vim。如果你还没装 VS Code，就让它选 Vim 别动；装了就选 Visual Studio Code。

- 调整 PATH 环境 **(Adjusting your PATH environment)**：确保选中 Git from the command line and also from 3rd-party software（这是推荐选项）。
- 选择终端模拟器：建议选 MinTTY。
- *再次强调：**选不明白就直接 Next**，不会爆炸的。*

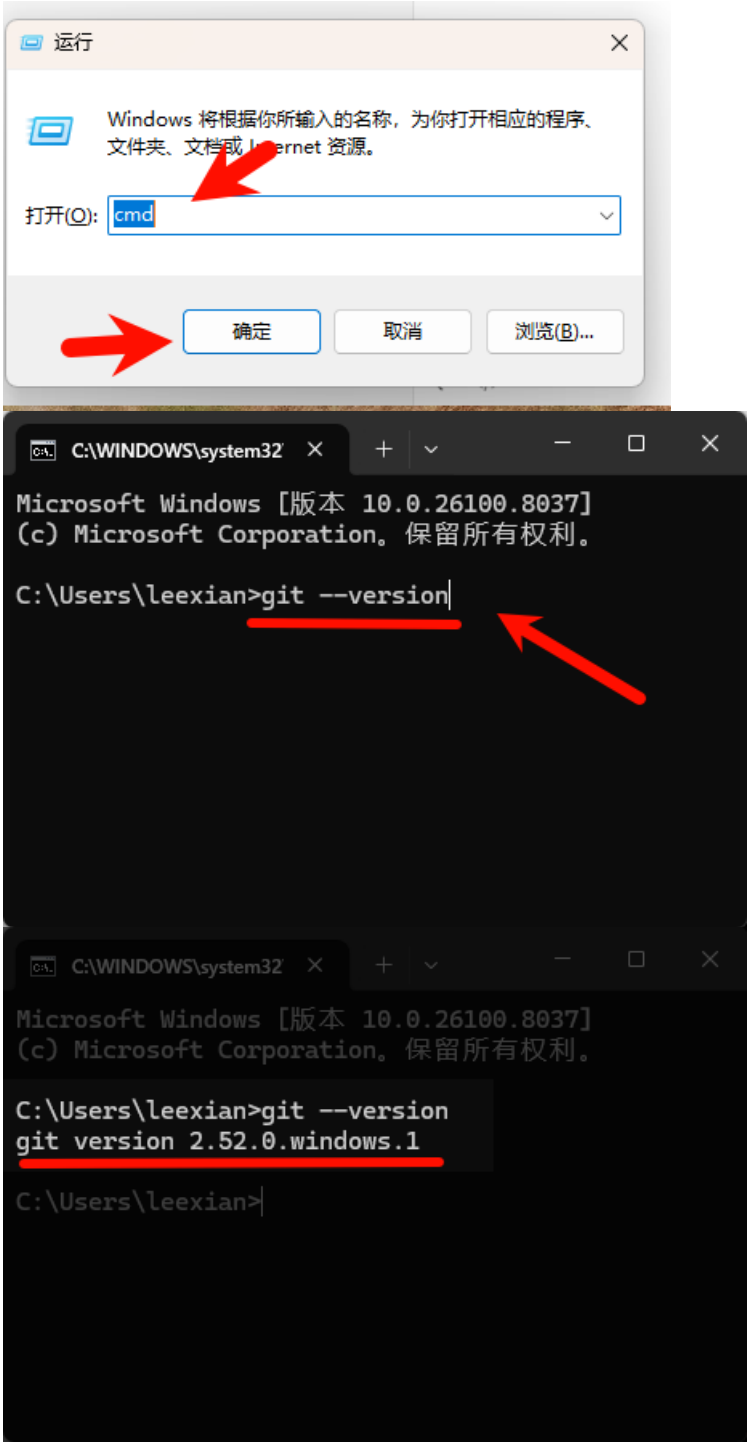
3. 验证安装是否成功

安装完成后，按下键盘上的 **Win + R** 组合键，打开"运行"对话框，输入 `cmd` 并回车，打开命令提示符窗口。在窗口中输入以下命令：

Bash

```
git --version
```

如果看到类似 `git version 2.x.x.windows.1` 的字样，说明你已经成功安装了！



第二步：配置 Git 用户信息（命令行用户必做）

如果你打算使用 **Git Bash（命令行）**，必须完成此步骤，否则 GitHub 无法将代码提交关联到你的账号。使用 Sourcetree 或 VSCode 等 GUI 工具的用户，通常可在软件设置中直接配置，可跳过本步。

> ⚠️ **注意：** 请务必使用你 **GitHub 账号**对应的用户名和邮箱，否则你的贡献将无法计入“绿墙”（Contribution Graph）。

在 **Git Bash** 中依次输入以下命令（每行输完按回车）：

1. **设置用户名**（将引号内替换为你的 GitHub 用户名，例如 `"jees996"`）：

   Bash

   ```
   git config --global user.name "你的用户名"
   ```

2. **设置邮箱**（必须是你的 **GitHub 注册邮箱**）：

   Bash

   ```
   git config --global user.email "你的注册邮箱"
   ```

如何确认成功？

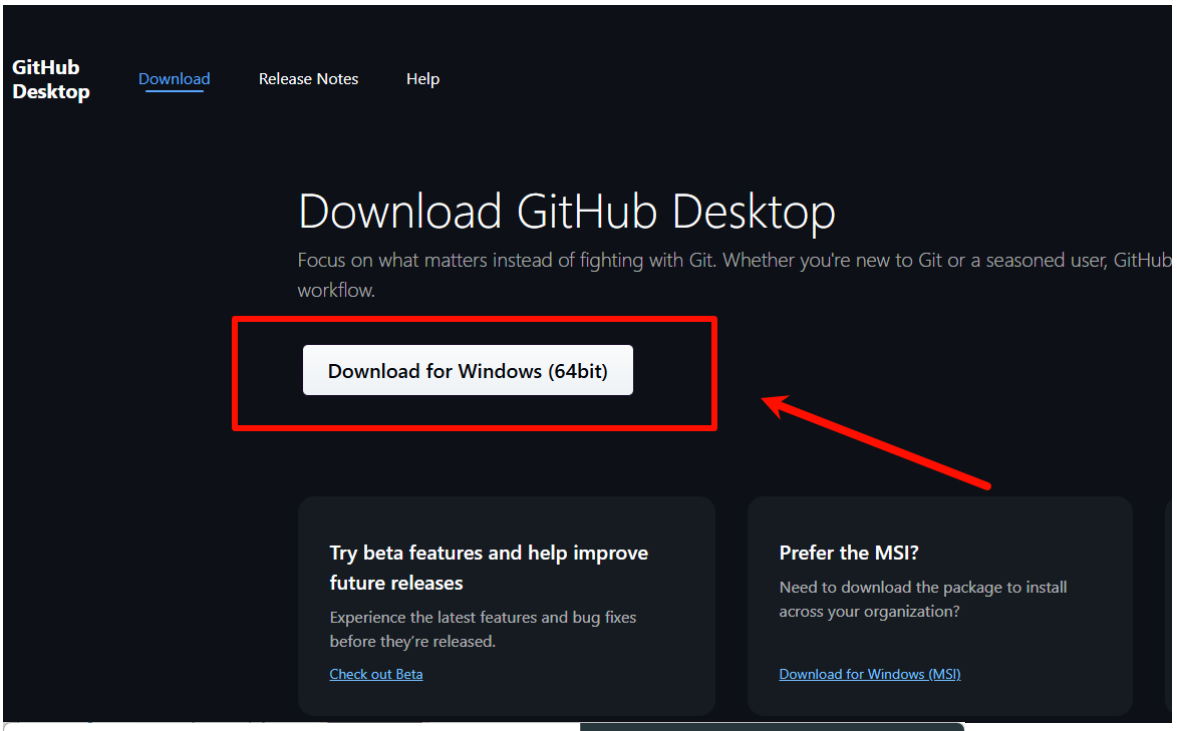
输入命令后若无任何报错提示（直接跳转到下一行），即表示配置成功。如果你只打算通过命令行使用 Git，到此基础配置便已完成。

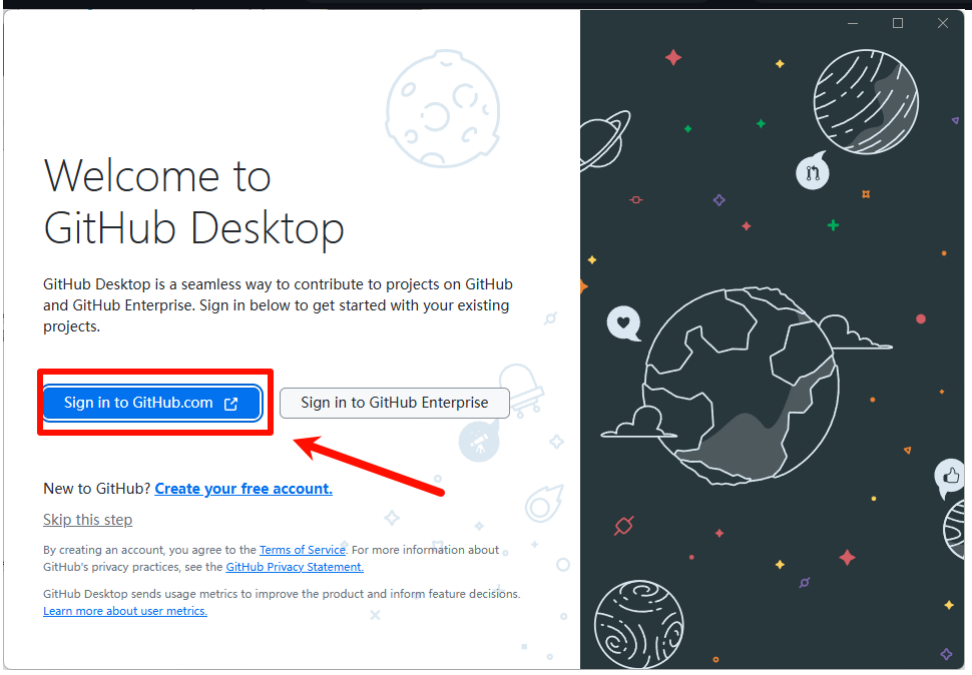
下载并安装 GitHub Desktop（图形化软件，新手推荐）

> 💡 **为什么用图形化软件？**
>
> - 不用记命令，点点鼠标就能操作
> - 可视化界面，仓库状态一目了然
> - 适合 Git 初学者快速上手

1. 下载安装

1. 访问官网：https://desktop.github.com/download/
2. 点击 **"Download for Windows"** 下载安装包(其他系统则找到对应系统安装包)
3. 双击安装包，等待自动安装完成
4. 打开 GitHub Desktop，使用你的 GitHub 账号登录





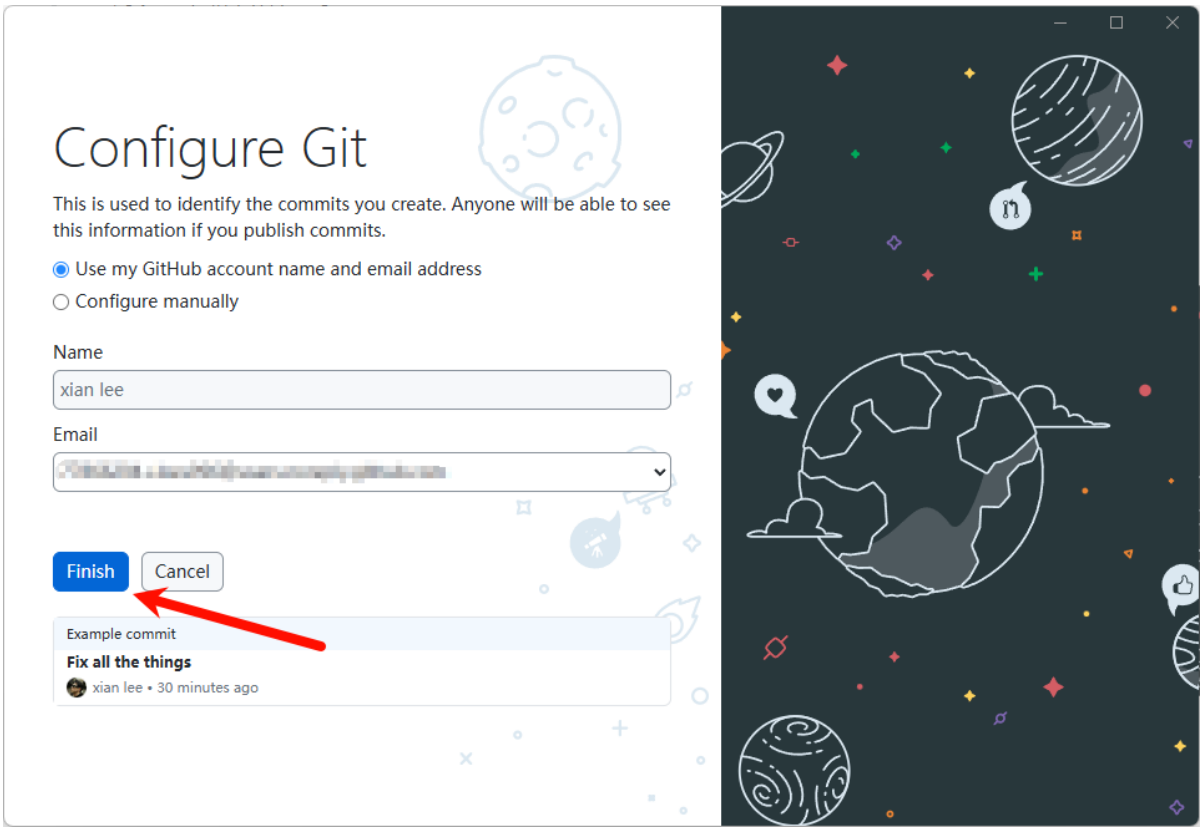
2. 首次配置

登录后会弹出配置窗口，填写以下信息：

- **Your Name**: 你的名字（随意填）
- **Email**: 你注册 GitHub 时用的邮箱（必须和你GitHub注册邮箱一致）

  你也可以勾选“Use my GitHub ...”就会自动调用你的账号信息

点击 **"Finish"** 完成配置。



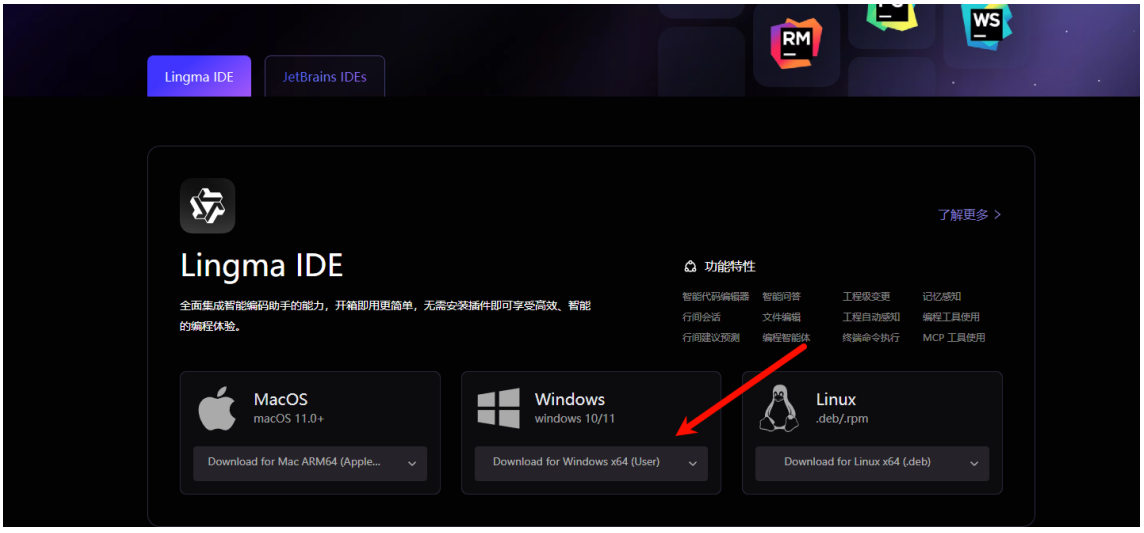
> ✅ **现在你已经安装好了 GitHub Desktop，接下来可以用图形化界面完成后续所有操作！**

安装并配置 AI 编程助手

AI 助手可辅助生成 Git 命令、解释报错并提供代码优化方案。

1. **推荐工具：通义灵码（免费）**

- **官网地址**：tongyi.aliyun.com/lingma
- **安装流程**：访问官网 -> 点击“免费下载” -> 选择对应编辑器版本 -> 运行安装包。



- **登录激活**：在编辑器（IDE）中点击插件图标，使用阿里云账号登录即可自动激活。
- **语言设置**：在插件设置中将语言切换为“简体中文”。

---

2. 其他备选工具

工具名称	官方网址	成本方案	核心优势
GitHub Copilot	github.com/features/copilot	$10/月（学生凭证免费）	GitHub 官方出品，集成度最高
Cursor	cursor.sh	基础版免费	基于 VS Code 的 AI 原生编辑器
Codeium	codeium.com	个人版免费	支持多种编辑器，响应迅速

---

3. 注意事项

- **辅助定位**：AI 仅为辅助工具，切勿完全依赖。
- **安全确认**：在执行 AI 生成的命令行之前，请务必核对命令的安全性，避免误操作。

所有工具已准备就绪。你想了解如何利用 AI 助手快速生成你的第一条 Git 提交命令吗？

使用教程

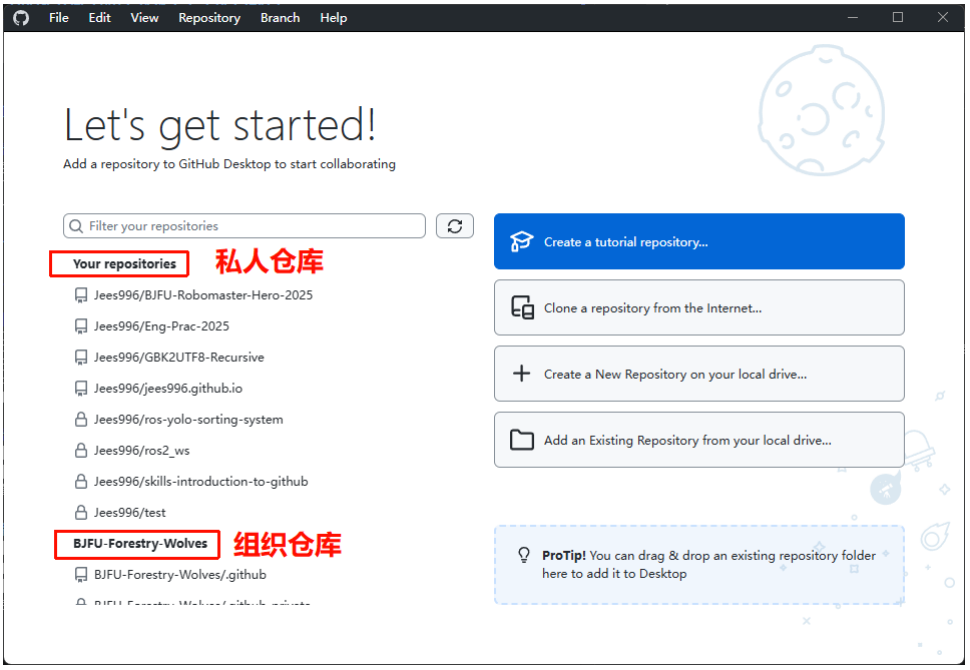
一、图形化界面（GUI）快速上手（新手推荐）

> **为什么先用 GUI？** 像用普通软件一样，点点鼠标就能操作 Git，无需记忆命令，5 分钟上手。

1.1 GitHub Desktop 首页认识

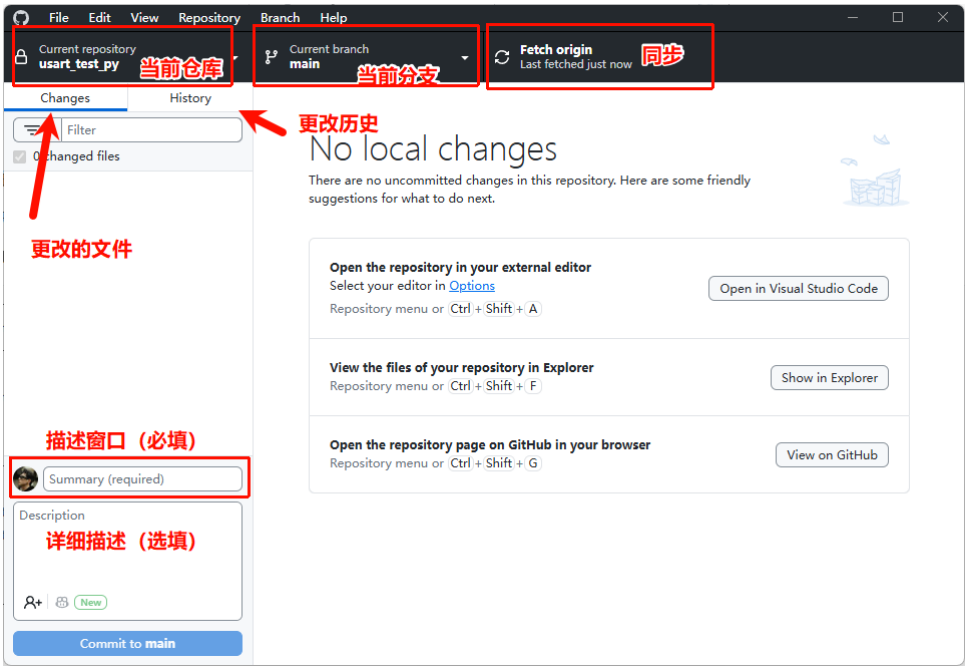
第一次打开 GitHub Desktop，你会看到这个界面：

这是初次使用的首页，看不懂先不着急，这里只做介绍。



打开仓库后的页面：

这是之后打开仓库后的页面，提前认识一下，后续会告诉你怎么创建或打开仓库



1.2 创建/克隆仓库

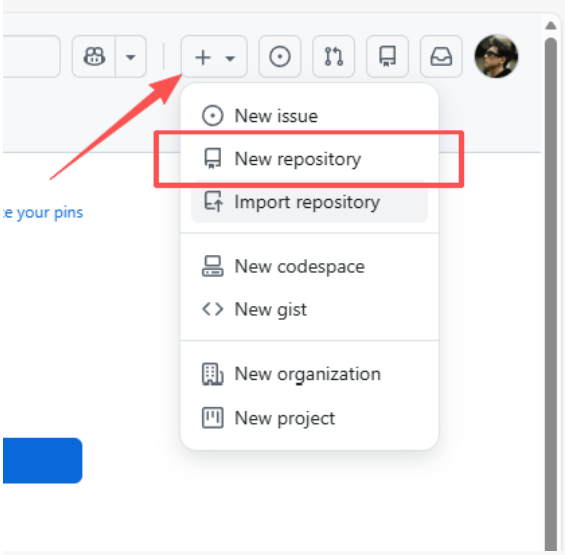
1.2.1 创建云端仓库

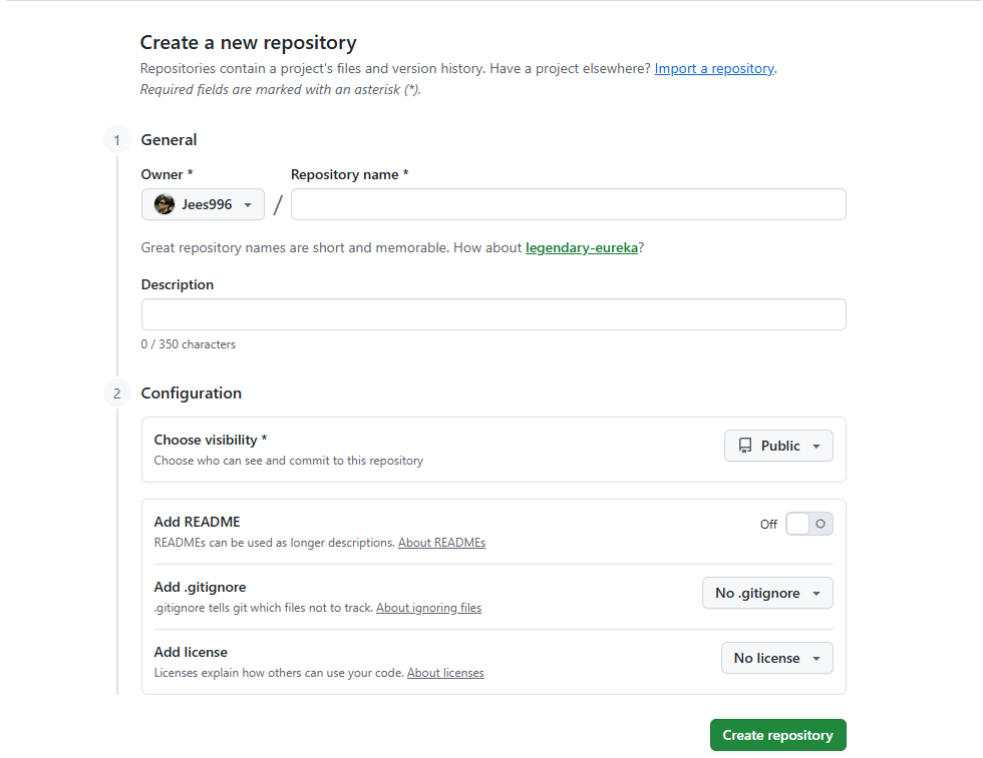
适用场景：不管你电脑上有没有任何代码，都可以先在 GitHub 上建好仓库，再把它下载（克隆）到电脑上开始写作业。

在 GitHub 官网建立新仓库

请打开 github.com 并登录，点击右上角的 **“+”** -> **“New repository”**（新建仓库），进入创建页面。

请对照你看到的页面（就是下面这张图），按以下标准填空：





GitHub 网页端：新建仓库配置

在 GitHub 官网点击 **“+” -> “New repository”** 后，请按以下标准快速配置：

1. 基础设置 (General)

- **Repository name**: 输入 `test`（必填，教程统一用名）。
- **Description**: 选填，如“我的第一个项目”。

2. 核心配置 (Configuration)

- **Visibility**: 必选 **Private** (私密)。
  - *理由：初学者建议“关门练功”，代码仅自己可见，安全感拉满。*
- **Add README**: **保持不勾选 (Off)**。
  - *理由：创建一个纯净的“空壳”，GitHub 才会显示后续步骤所需的“代码小抄”。*
- **.gitignore / License**: 均保持默认 **None**。
  - *备注：新手阶段暂不需要，后续项目复杂时可再添加。*

3. 完成创建

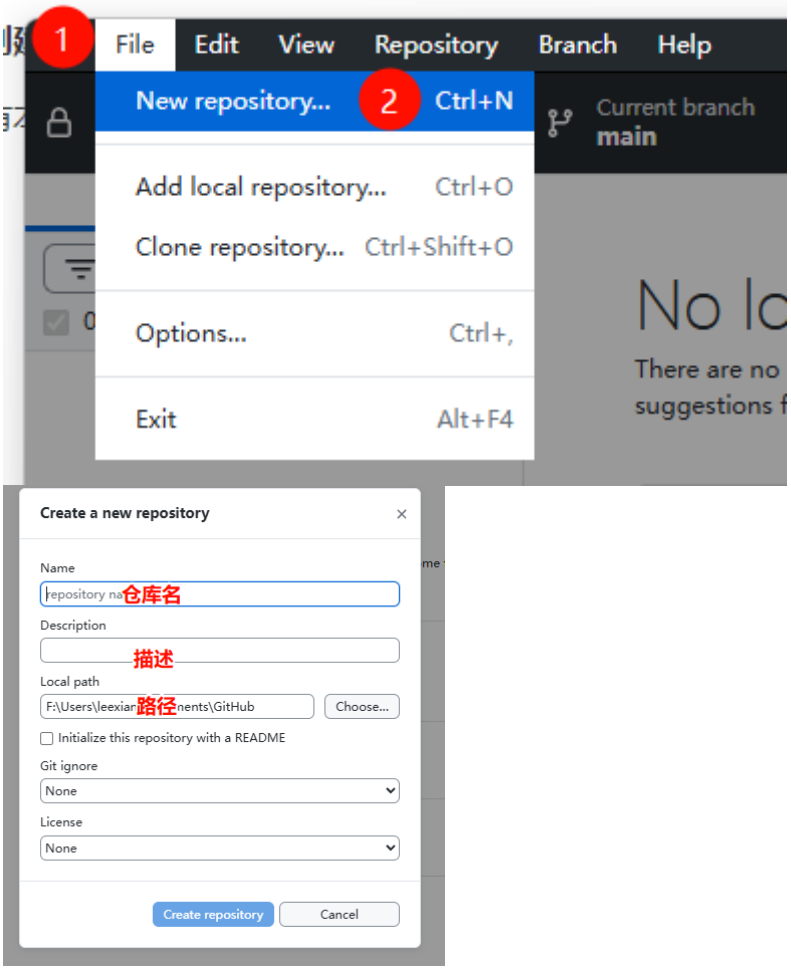
- 点击右下角绿色按钮 **Create repository**。

1.2.2 创建本地仓库并发布到云端

适用场景：

- 你已经在电脑上写好了代码（比如课程作业、项目练习）
- 现在想把这些代码备份到 GitHub 云端
- 或者想和同学协作开发，需要共享代码
- 简单说：**本地有文件 → 发布到 GitHub**

请打开GitHub Desktop，点击右上角的 **“file”** -> **“New repository”**（新建仓库），进入创建页面。



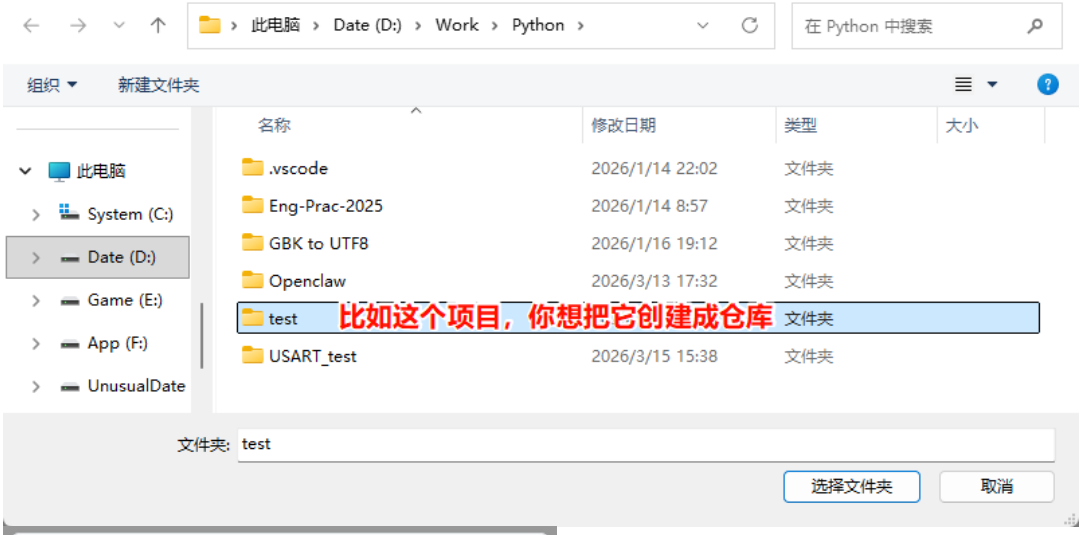
此页面用于在你的电脑上直接创建一个受 Git 管理的项目文件夹。

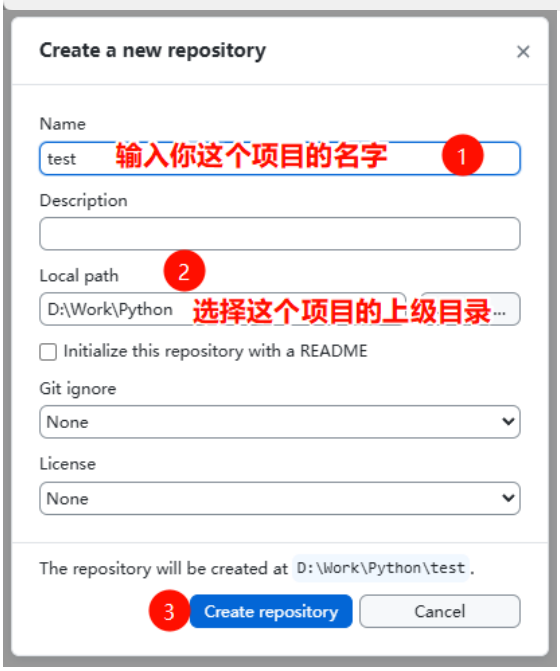
- **Name (仓库名)**：项目的名称，建议使用英文（如 `my-project`）。
- **Description (描述)**：可选填，简述项目用途。
- **Local path (路径)**：项目文件夹在电脑里的存放位置。点击 **Choose** 可更改。
- **Initialize with a README**：勾选后会自动生成一个说明文件。若打算关联已有的 GitHub 仓库，建议先不勾选。
- **Git ignore**：选择忽略名单模板（如 `STM32` 或 `Python`），防止上传无用的系统缓存或临时文件。

- **License**：选择开源许可证（如 `MIT`），规定他人是否可以商用或修改你的代码。

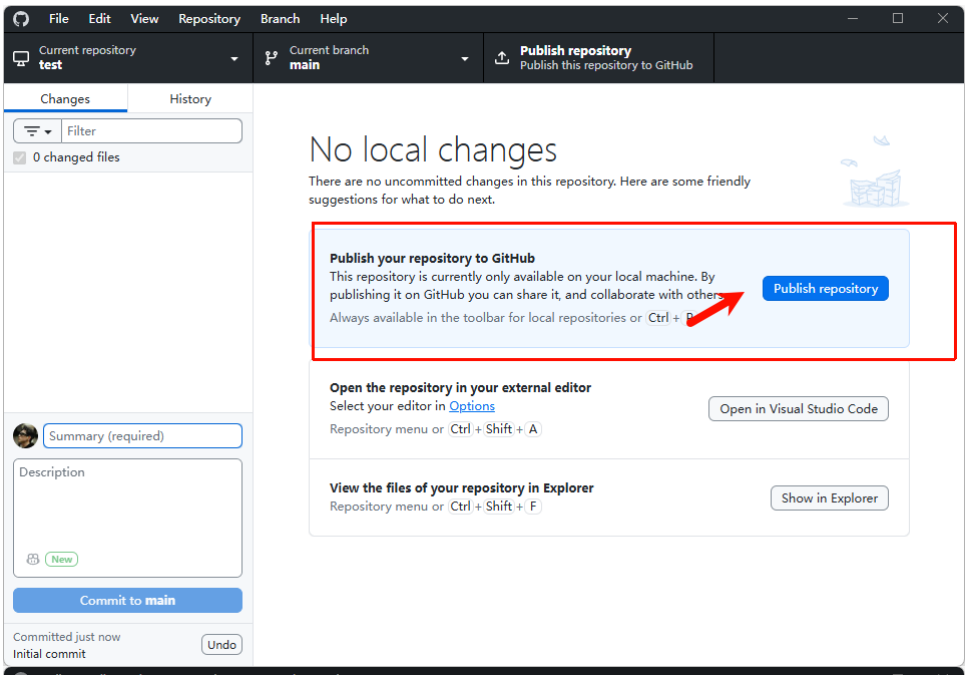
操作建议：填好 **Name** 并确认 **Local path** 无误后，点击 **Create repository** 即可完成创建。

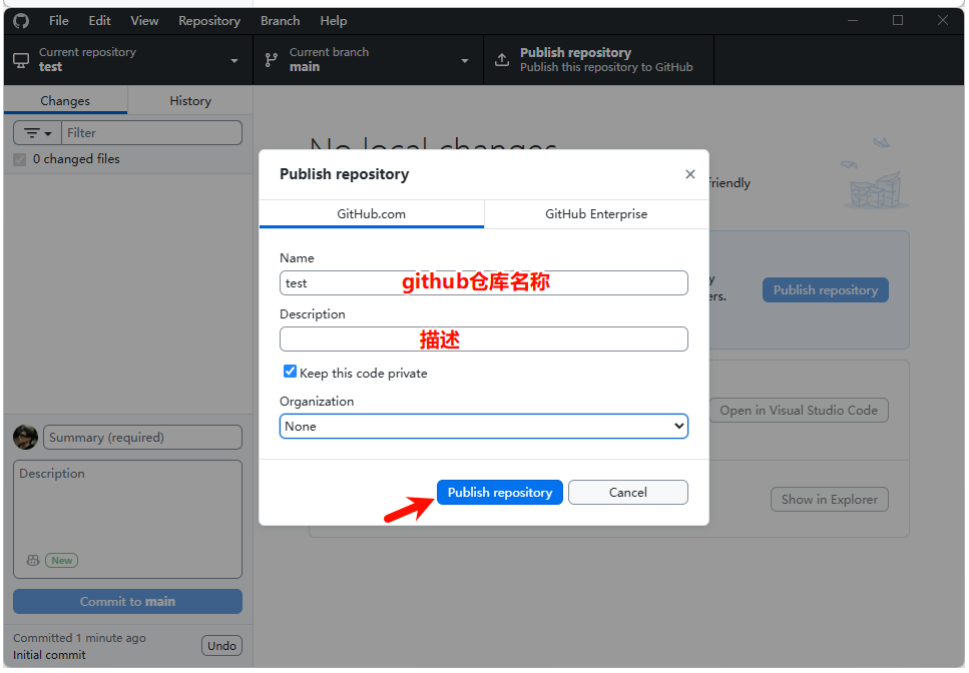
这里我演示的是：你本地有项目（代码），现在把你的项目创建成仓库





把你的本地项目发布到github：





1.2.3 克隆他人或云端仓库（最常用）

适用场景：

- 你想下载别人的开源项目来学习
- 你在 GitHub 上创建了新仓库，现在要下载到电脑上开始写代码
- 你和同学协作开发，需要把他的代码拉取到你电脑上
- 简单说：**从 GitHub 下载 → 到本地电脑**

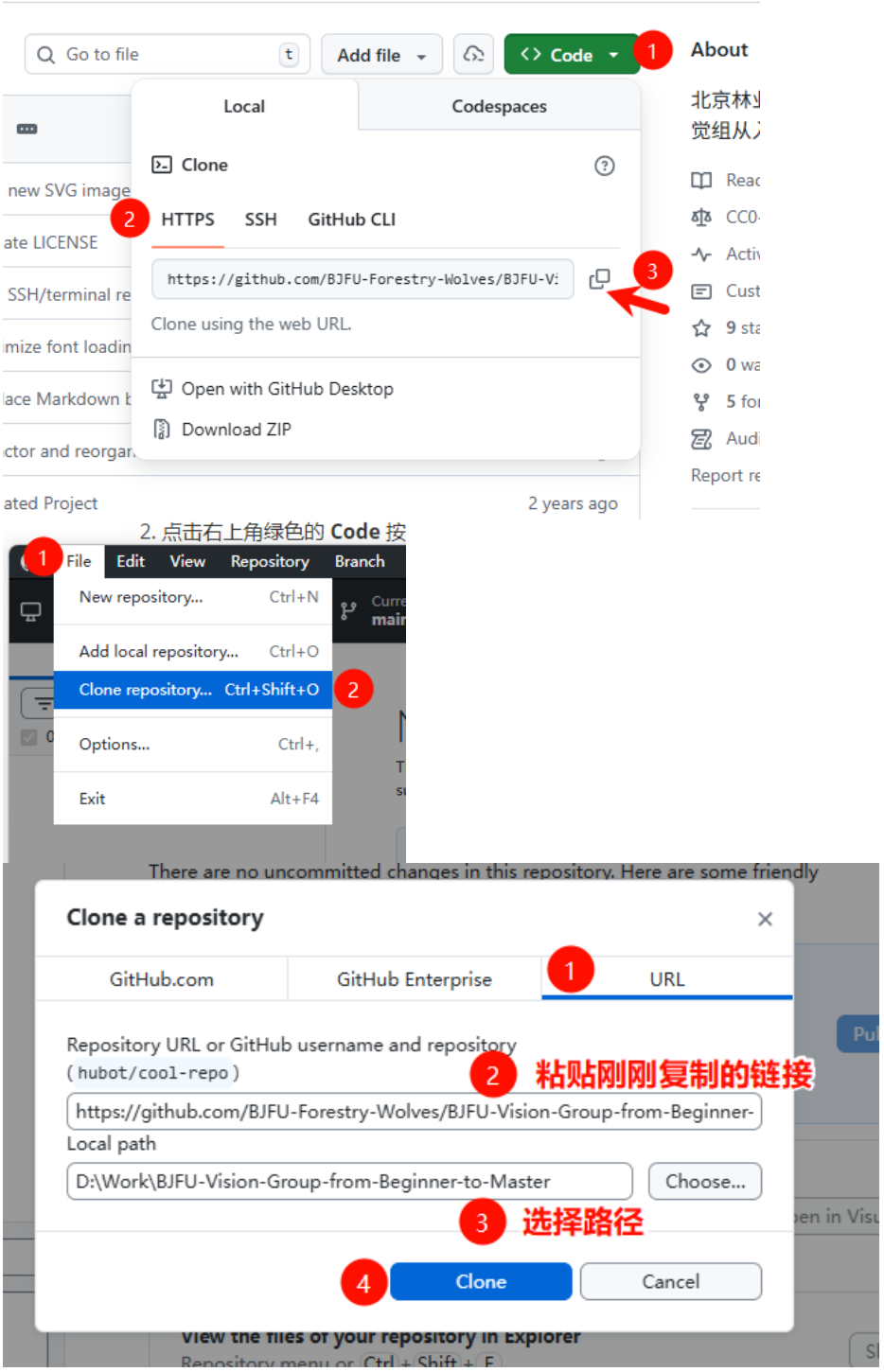
克隆他人/自己的项目

1. 在浏览器中打开 GitHub，找到你想要的项目页面
2. 点击右上角绿色的 **Code** 按钮
3. 复制显示的仓库链接（HTTPS 格式）
4. 打开 **GitHub Desktop**

5. 点击 **File → Clone repository...**
6. 切换到 **URL** 标签页
7. 粘贴刚才复制的链接
8. 选择本地保存路径
9. 点击 **Clone** 开始下载

此处以北京林业大学森林狼战队视觉组开源文档为例：

项目网址：https://github.com/BJFU-Forestry-Wolves/BJFU-Vision-Group-from-Beginner-to-Master

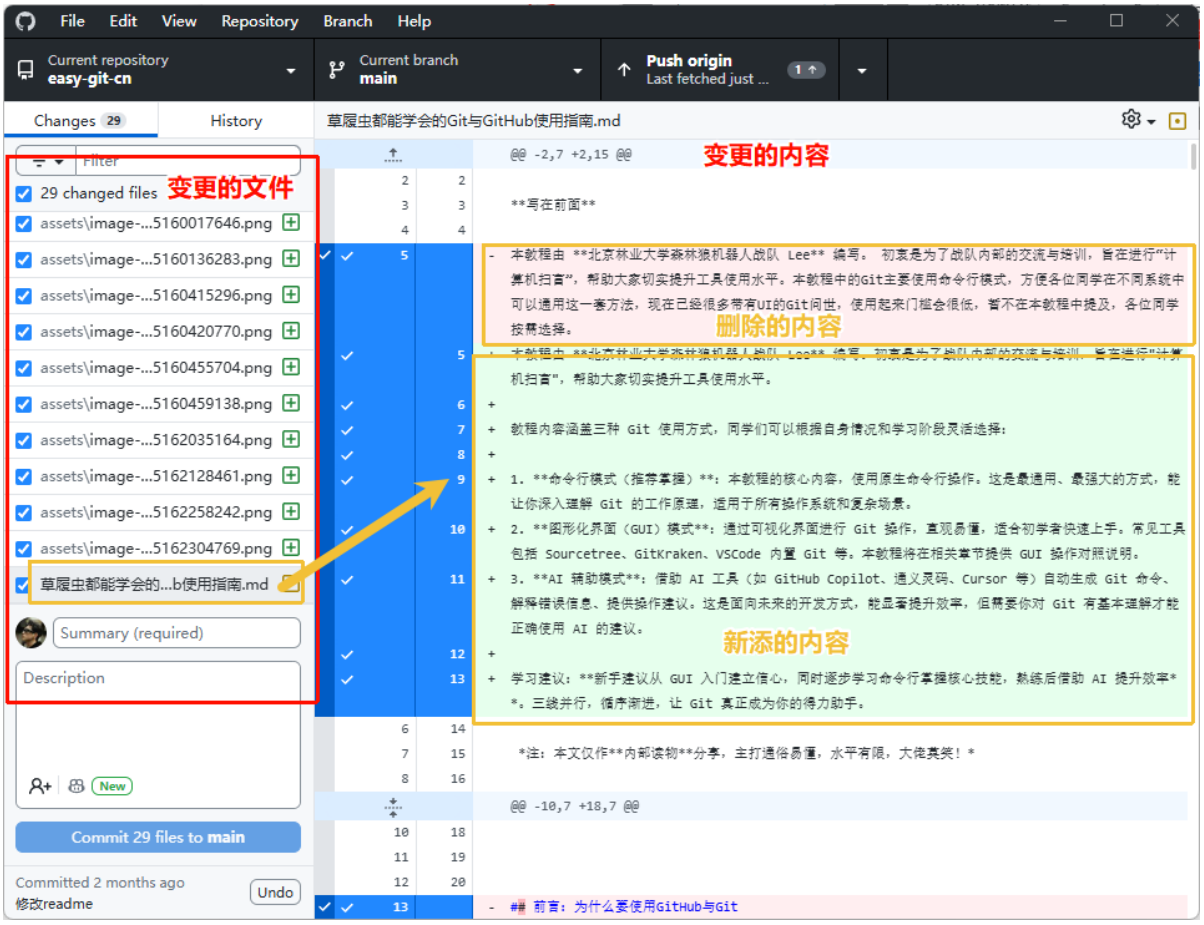


💡 **常见问题：**

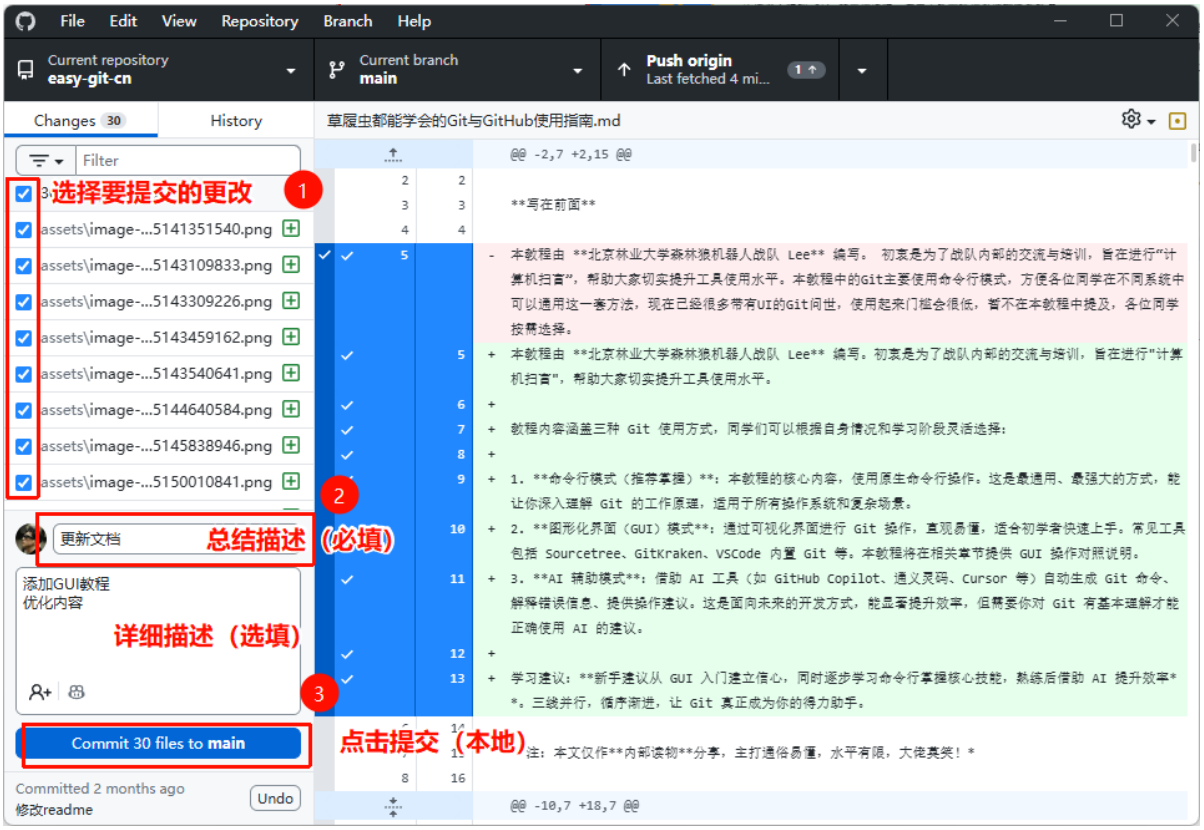
- **克隆失败怎么办？** 检查网络连接，确保能访问 GitHub
- **速度太慢？** 可以尝试切换网络环境或使用代理
- **克隆后找不到文件？** 去你选择的 Local path 文件夹里查看

1.3 查看文件变更

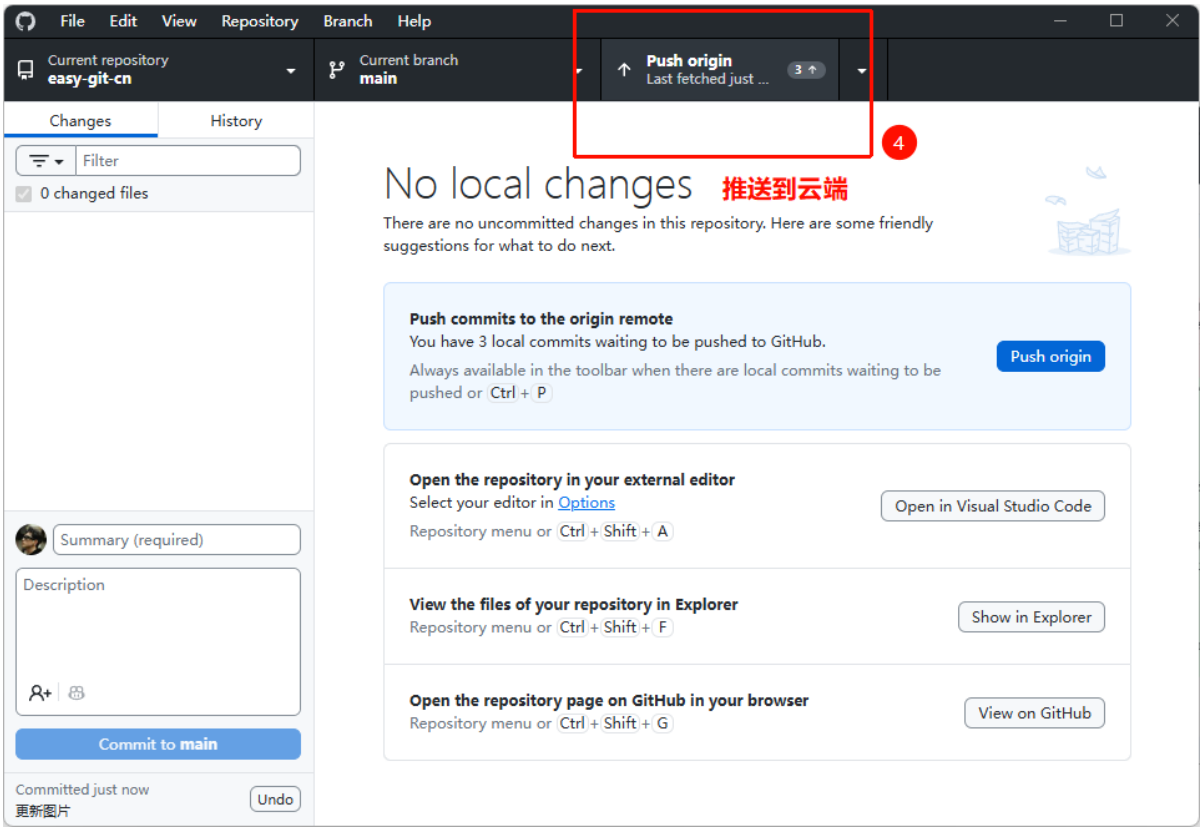
如果你更改了你这个项目中的某个文件，可以在**GitHub Desktop**中看到你更改了什么文件。



1.4 提交代码



1.5 推送到远程



1.6 其他操作后续补充

1.7 常用 GUI 工具对比

- GitHub Desktop（官方出品，最简单）
- Sourcetree（功能强大）
- VSCode 内置 Git（编辑器集成）

二、AI 辅助提效（现代开发方式，后续补充）

> **AI 能帮你做什么？** 生成命令、解释错误、提供建议，但需要你有基本判断力。

2.1 通义灵码使用指南

- 下载安装与配置
- 让 AI 生成 Git 命令
- 解读报错信息

2.2 其他 AI 工具

- GitHub Copilot（付费，学生免费）
- Cursor（AI 原生编辑器）

三、命令行基础教程（必须掌握）

> **为什么要学命令行？** 通用性强、功能最全、远程必备、理解原理。

3.1 Git 基础概念（先了解）

- 什么是仓库、工作区、暂存区
- 分支是什么
- HEAD 是什么

3.2 创建仓库（从零开始）

3.2.1 本地无文件，从 GitHub 创建全新仓库

- 在 GitHub 官网建立新仓库
- 使用快捷命令一键完成本地初始化与推送（见后面教程）

3.2.2 本地有文件，绑定到 GitHub

- 检查本地是否已是 Git 仓库
- 三步绑定远程仓库并推送

3.2.3 下载他人现成的仓库

- git clone 命令详解
- 实战：克隆北京林业大学视觉组开源文档

3.3 日常开发核心指令（高频使用）

3.3.1 三板斧：Add → Commit → Push

- `git add .`：添加到暂存区
- `git commit -m "备注"`：提交到本地
- `git push`：推送到云端

3.3.2 查看状态与历史

- `git status`：检查工作区状态
- `git log --oneline`：查看提交历史

3.3.3 版本管理

- `git reset --hard [ID]`：版本回退
- `git pull`：拉取队友代码

3.3.4 分支操作

- `git branch -M main`：重命名分支
- `git branch`：查看/创建分支
- `git checkout`：切换分支

3.4 常见问题自救指南

- 推送报错：`! [rejected] ... (fetch first)`
- 分支不一致：master vs main
- 网络连接失败：Connection was reset

3.5 常用指令速查表

指令	功能	备注
git init	初始化仓库	
git clone	克隆远程仓库	
git add	添加文件	
git commit	提交代码	

指令	功能	备注
git push	推送云端	
git pull	拉取代码	
git status	查看状态	
git log	查看历史	
git reset	版本回退	⚠️危险操作
git branch	分支管理	

1、如何创建仓库（基础）

本章将带你上手 Git 的核心基础。首先，我们需要解决“仓库从哪来”的问题，通常有两种方式：**自己新建**或**从网上下载**。

本教程将基于三种实际情况进行分别演示：

1、本地没有文件，使用GitHub创建全新的仓库

2、本地有文件，在GitHub创建新仓库并绑定

3、下载他人现成的仓库

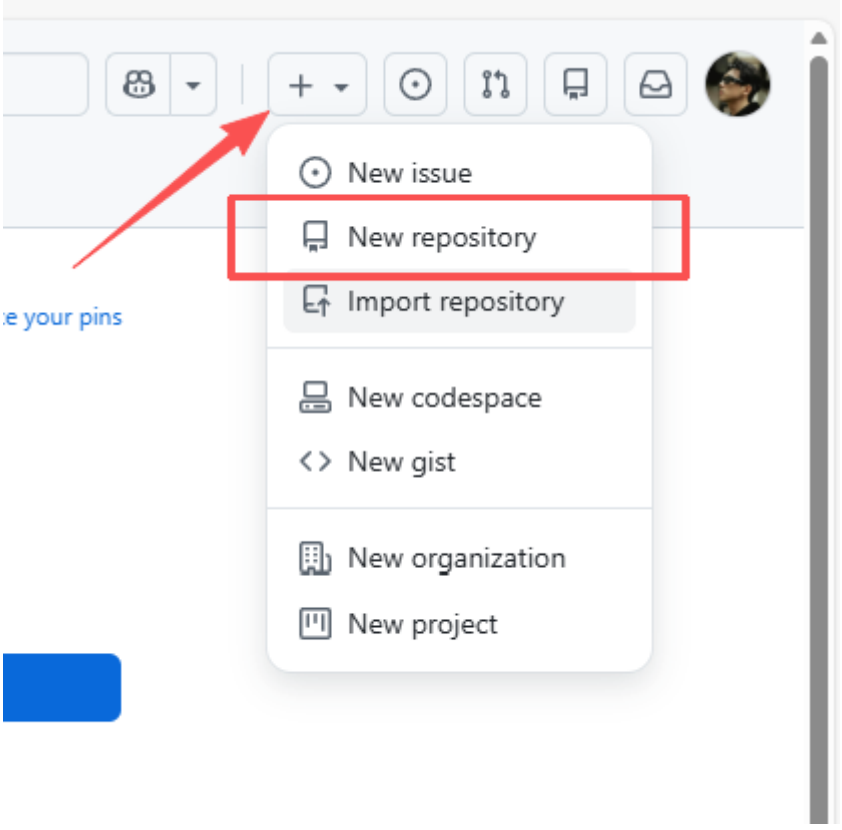
1、自己创建一个新的仓库

第一步：在 GitHub 建立新仓库

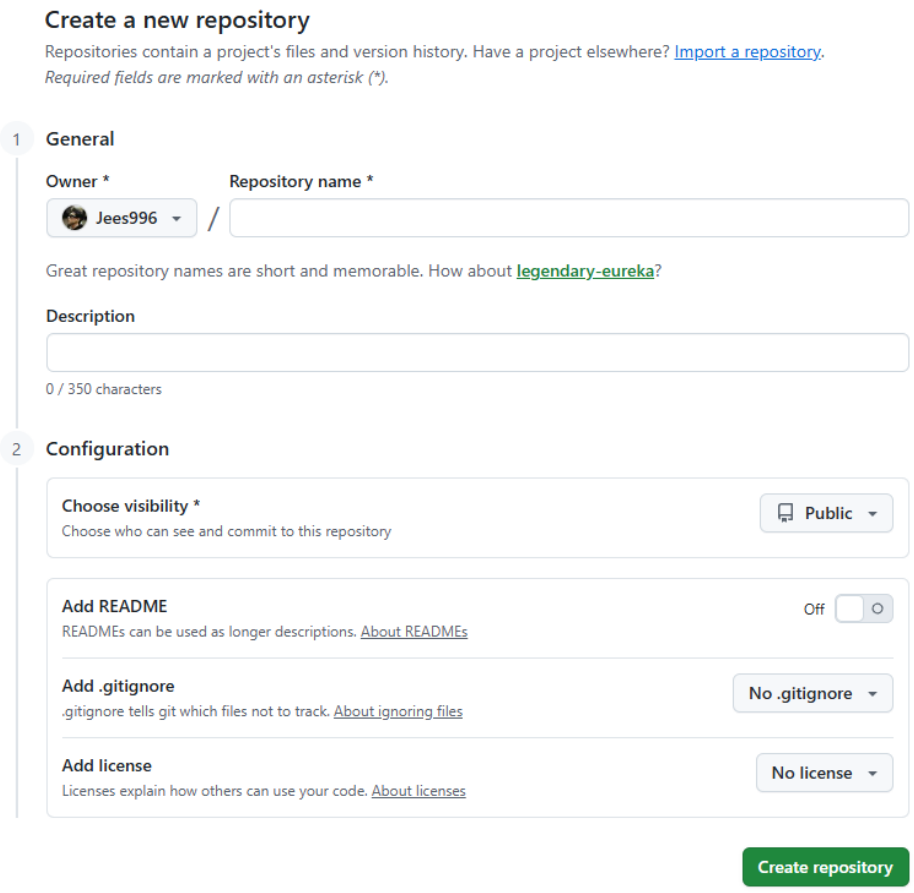
适用场景：不管你电脑上有没有任何代码，都可以先在 GitHub 上建好仓库，再把它下载（克隆）到电脑上开始写作业。

1. **第一步：在 GitHub 官网建立新仓库**

   请打开 github.com 并登录，点击右上角的 **“+”** -> **“New repository”**（新建仓库），进入创建页面。



请对照你看到的页面（就是下面这张图），按以下标准填空：



1. General (常规设置)

- **Owner (拥有者)**：默认是你自己，不用改。

- **Repository name (仓库名)**：输入 `test`。
  - *这是必填项，为了后续教程统一，请务必叫这个名字。*
- **Description (描述)**：选填，可以写“我的第一个测试项目”。

2. Configuration (配置选项)

- **Choose visibility (选择可见性)**：
  - **【关键】** 请手动选择 **Private (私密)**。
  - *理由：作为草履虫入门，我们先关门练功，只有你自己能看到代码，安全感拉满。*
- **Add README (添加说明文件)**：
  - **【注意！关键点】 请保持默认 Off (不要勾选/灰色)。**
  - *理由：千万别勾！我们要创建一个纯净的“空壳子”，这样 GitHub 才会显示出那个教我们怎么上传代码的“作弊小抄”页面（就是下面要用的代码块）。这个“readme文件你可以后续自己创建”*
- **Add .gitignore**：保持默认 **No .gitignore**。

  是什么？ 它是给 Git 看的**“垃圾屏蔽名单”**。写在里面的文件（比如程序运行时产生的临时缓存、编译生成的乱七八糟的中间文件），Git 会自动无视它们，只上传代码源文件。

  备注： **新手先不管**。以后等你项目复杂了，不想上传垃圾文件时再加也不迟。
- **Add license**：保持默认 **No license**。

  是什么？ 它是给人类（和律师）看的**“法律声明”**。规定别人能不能抄你的代码、能不能拿去卖钱（比如著名的 MIT 协议、GPL 协议）。

  备注： **新手先不管**。

3. 完成创建

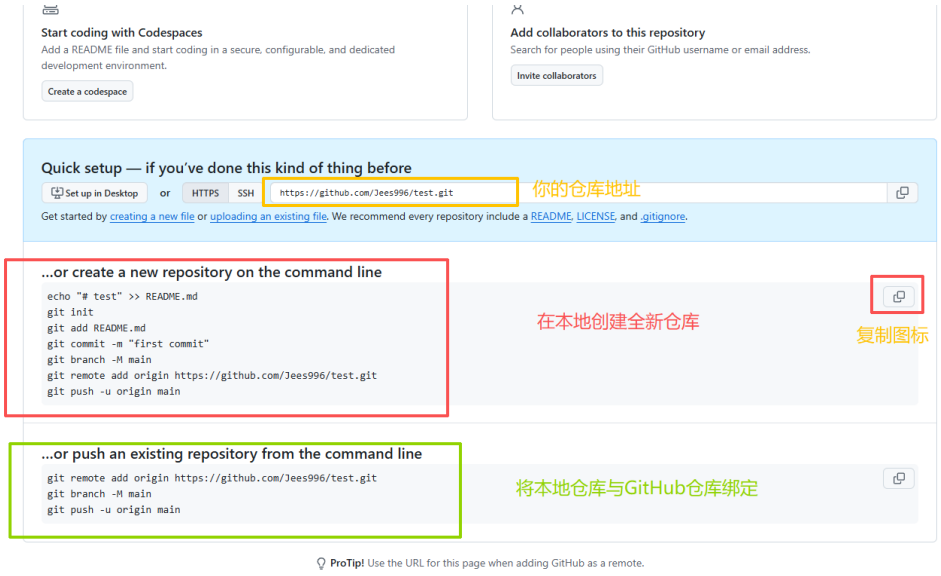
- **Create repository**：确认以上信息无误后，点击页面右下角的绿色按钮 **“Create repository”**。

第二步：在本地创建仓库并与GitHub绑定

创建成功后，你不会看到文件列表，而是会看到下面这个页面。

这是 GitHub 怕你不会用，贴心地给你准备的 **“一行代码都不用自己写”** 的复制粘贴区，以帮助你完成本地仓库的建立与绑定：

（这个页面是你没选择生成任何文件才会出现，如果没有这个页面可以重新创建一次，或者手动输入命令也可以）



这时候请进入你想要存放项目的文件夹（比如 `D:\Work\stm32\test`），然后根据你的情况，二选一：

情况 A：本地没有文件，创建全新仓库

...or create a new repository on the command line

适用场景：你电脑上也是空的，想从零开始建仓库。

第一步：准备终端 请在电脑上打开 **Git Bash**，输入 `cd /d/Work/stm32/test`（或者直接在 `D:\Work\stm32\test` 文件夹里右键打开 Git Bash）。

第二步：一键执行 点击 GitHub 页面代码块右上角的 **复制图标**（或手动复制下面这坨代码），**一次性粘贴**到你的 Git Bash 黑框里，按回车执行。

Bash

```bash
echo "# test" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/Jees996/test.git
git push -u origin main
```

【成功标志】 只要黑框框最后没有出现 `Error` 字样，并且看到了 `Branch 'main' set up to track remote branch...` 的提示，说明你的本地代码已经成功上传到 GitHub 了！

代码功能速查表 如果不明白这坨代码在干什么，请对照下表，这些常用指令都会在后续讲到，不看也行。

代码指令	核心功能	详细说明
`echo "# test" >> README.md`	**创建文件**	新建一个名为 `README.md` 的文件，并写入内容。Git 无法追踪空文件夹，必须有文件才能初始化。
`git init`	**初始化仓库**	在当前目录下生成 `.git` 隐藏文件夹，使其成为一个受 Git 管理的本地仓库。
`git add README.md`	**添加到暂存区**	将新创建的文件加入 Git 的追踪列表（暂存区），准备提交。
`git commit -m "..."`	**提交到本地**	将暂存区的内容正式归档，生成版本历史中的第一个“节点”。
`git branch -M main`	**重命名分支**	**关键步骤**。将本地默认分支（通常叫 `master`）强制重命名为 `main`。
`git remote add origin ...`	**绑定远程地址**	将本地仓库与 GitHub 上的云端仓库建立连接，并将云端地址命名为 `origin`。
`git push -u origin main`	**推送到云端**	将本地 `main` 分支的代码上传到远程 `origin` 地址，并建立追踪关系。

情况 B：本地有文件，将本地仓库与GitHub仓库连接

适用场景：**“补票上车”。** 你的电脑本地已经有一个 Git 仓库了（也就是说，你之前已经写了代码，并且敲过 `git init`、`git add`、`git commit` 这一套连招了），现在只是想把它关联到 GitHub 上保存起来。

🛑 **动手前的“资格审查” (必看！)**

这串代码能运行的绝对前提是：你本地必须已经是一个 Git 仓库！

怎么知道是不是？ 请在 Git Bash 黑框里输一下 `git status`。

- ✅ **显示** `On branch ...`：恭喜，资格符合，请继续往下看。
- ❌ **报错** `fatal: not a git repository...`：说明你还没“买票”！**请直接跳回上面的（情况 A）**，先把 `git init`、`add`、`commit` 那一套指令全部运行一遍，再来这里。

第一步：一键执行

确认本地没问题后，复制下面这三行代码（把**仓库地址**换成你自己的），粘贴到 Git Bash 黑框里，回车！

车！

Bash

```
git remote add origin https://github.com/Jees996/test.git
git branch -M main
git push -u origin main
```

代码功能速查表 如果不明白这坨代码在干什么，请对照下表（这些常用指令都会在后续讲到，不看也行）：

代码指令	核心功能	详细说明
`git remote add origin ...`	**绑定远程地址**	将本地仓库与 GitHub 上的云端仓库建立连接，并将云端地址命名为 `origin`。
`git branch -M main`	**重命名分支**	**关键步骤。**将本地默认分支（通常叫 `master`）强制重命名为 `main`，防止推送到云端时因为名字对不上而报错。
`git push -u origin main`	**推送到云端**	将本地 `main` 分支的现成代码直接打包上传到远程 `origin` 地址，并建立追踪关系。

💡 常见报错自救

Q：我粘贴后报错 `fatal: remote origin already exists.` 怎么办？

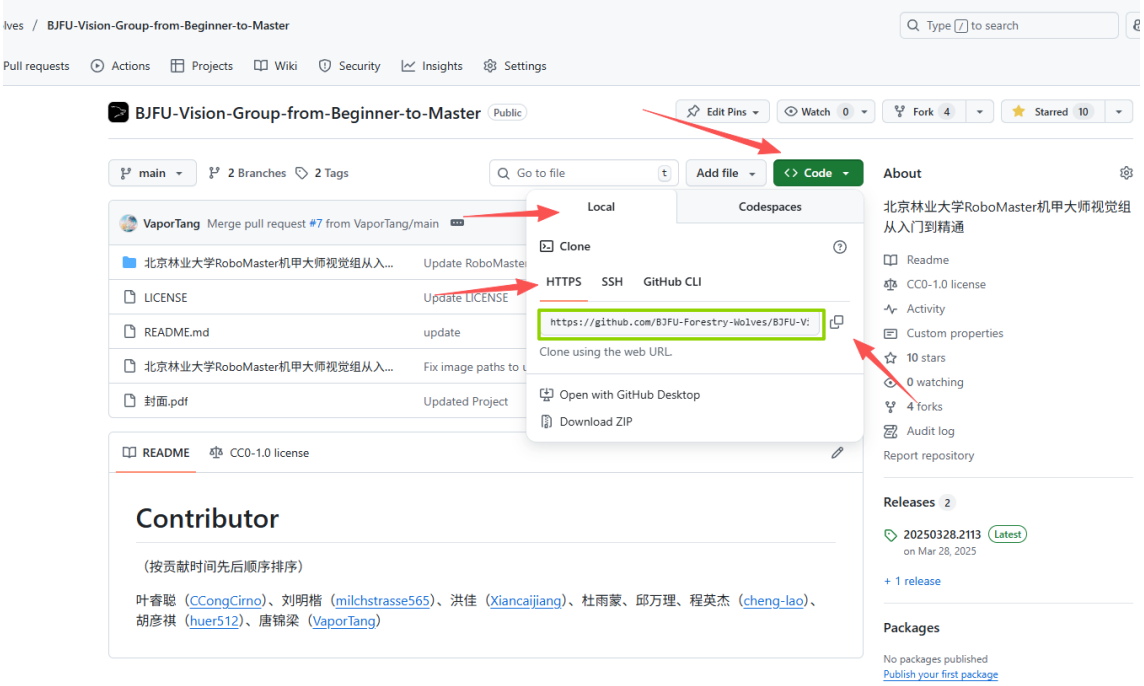
- **人话**：Git 说“你之前已经存过一个叫 origin 的号码了，不能重复存”。
- **解决**：说明你以前可能乱绑过地址。输入 `git remote remove origin` （删除旧号码），然后重新执行上面的第一行代码即可。

2、从 GitHub 下载他人仓库（`git clone`）

场景：要把“北京林业大学 RoboMaster 视觉组”的资料下载到电脑上。

- **第一步：复制地址** 进入我们队伍的GitHub，找到视觉组的这个仓库，仓库链接：https://github.com/BJFU-Forestry-Wolves/BJFU-Vision-Group-from-Beginner-to-Master.git

  （因为这个项目是开源的，所以以此为例，其他项目也一样通用。队伍内有一些仓库是闭源的，比如英雄的代码，想要下载需要先绑定你的Git和GitHub账号，后面再说）
- 复制目标仓库链接： `https://github.com/BJFU-Forestry-Wolves/BJFU-Vision-Group-from-Beginner-to-Master.git`

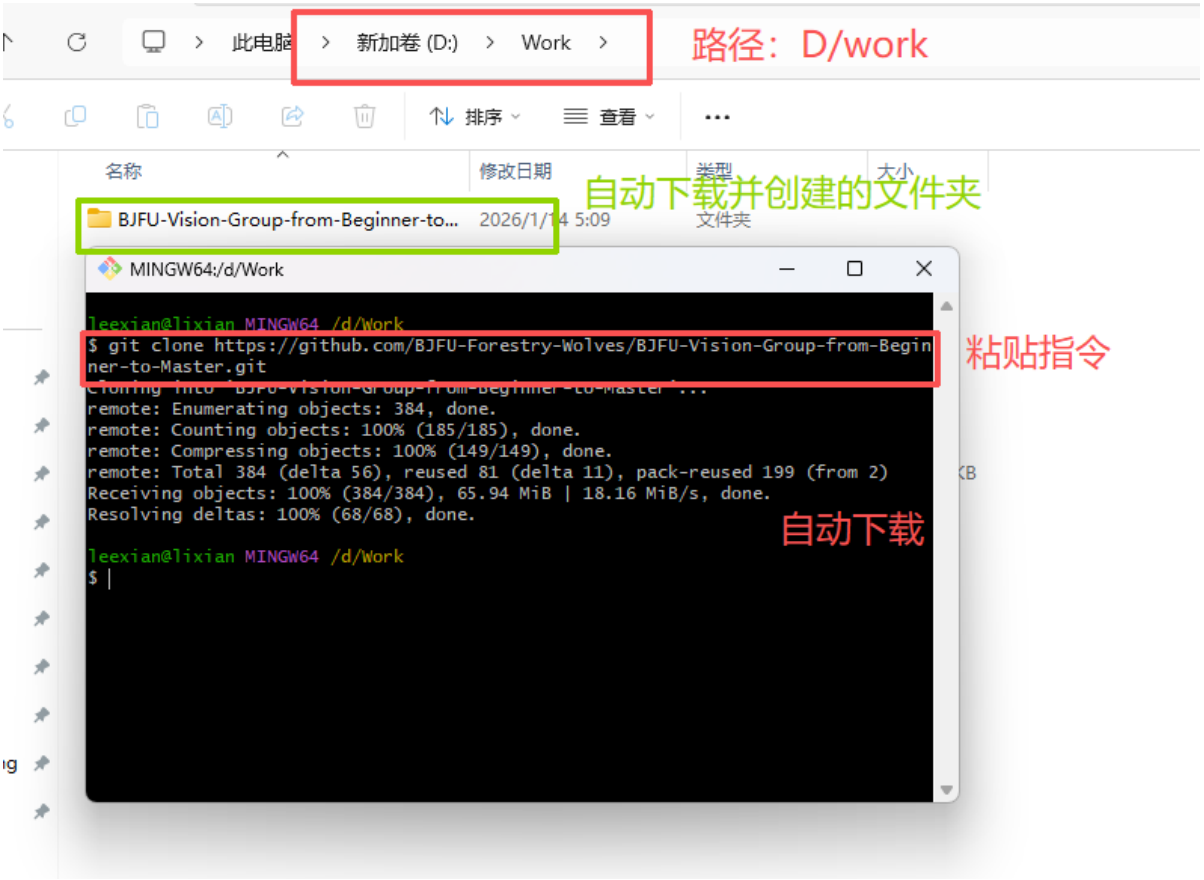


- **第二步：选择存放位置** 进入你想存放该项目的**父级文件夹**（例如 `D:\work`），在空白处**右键** -> **"Git Bash Here"**。*（注意：不需要手动新建空文件夹，Git 会自动建。）*
- **第三步：执行下载** 输入 `git clone`，空格，然后**右键选择 Paste**（或按 `Shift + Insert`）粘贴地址。 完整命令如下：

  Bash

  ```
  git clone https://github.com/BJFU-Forestry-Wolves/BJFU-Vision-Group-from-Beginner-to-Master.git
  ```

  按回车执行。
- **第四步：等待完成** 当看到 `100%` 且光标恢复输入状态时，下载结束。你可以去文件夹里看到下载好的项目了。



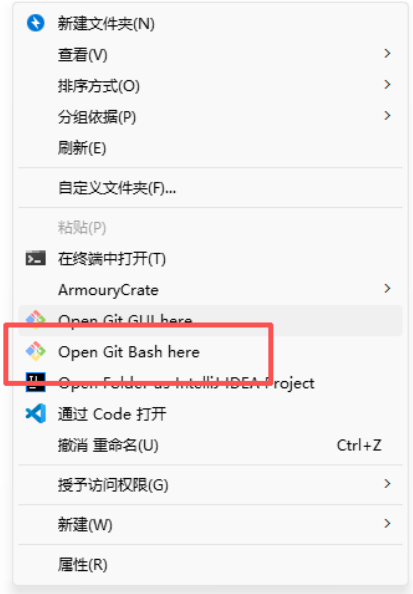
补充教程：初始化本地仓库

1、在本地新建仓库（`git init`）

场景：你有一个现成的项目文件夹 `D:\work\stm32`，想用 Git 管理它。

- **第一步：进入目录** 打开文件管理器，进入 `D:\work\stm32` 文件夹。
- **第二步：打开终端** 在文件夹空白处**右键**，点击 **"Git Bash Here"**。

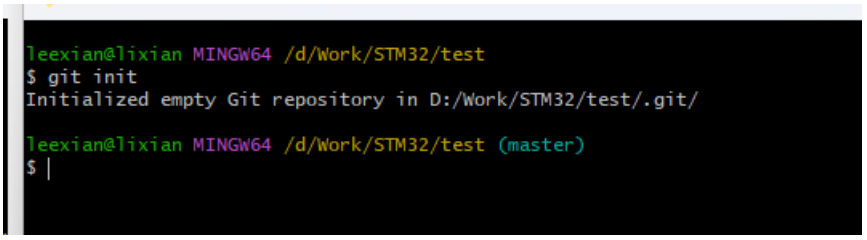
-



- **第三步：初始化** 在黑框中输入以下命令并回车：

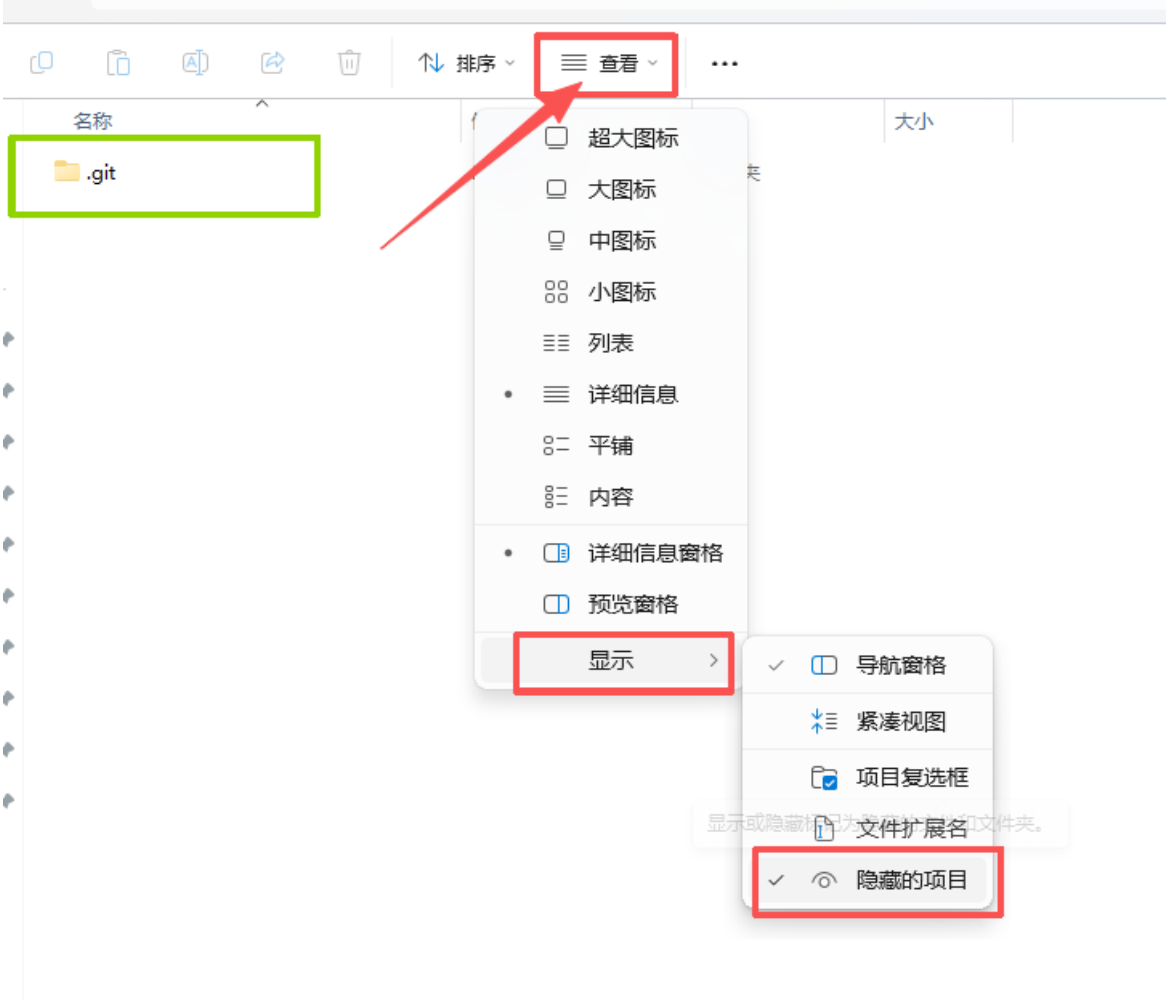
  Bash

  ```
  git init
  ```



- **第四步：验证**
  - 命令行路径后出现 `(master)` 或 `(main)` 字样。
  - 文件夹内自动生成了一个隐藏的 `.git` 文件夹（不要删除，不要修改）。
  - **完成。**

    查看隐藏的文件夹（只是演示，可以不打开这个设置）



四、常用基础指令（日常开发）

本章将介绍 Git 在日常工作中最高频使用的指令。只要学会第一节的这三个命令，你就能应对 90% 的开发场景。

注意：请按照上一章教程创建仓库，不然可能会出现比如分支名不一致等问题

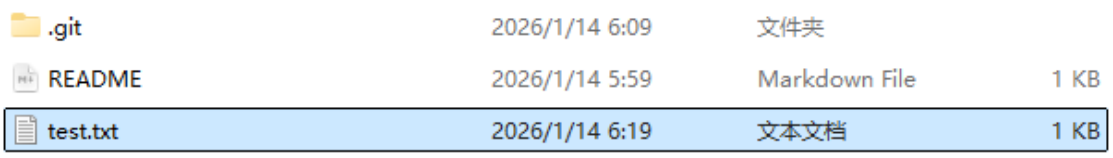
1 、三条核心指令：Add, Commit, Push

这三个指令的操作逻辑非常符合直觉，你可以把它想象成**网购**的全过程：

1. `add`：把想买的商品放进**购物车**。
2. `commit`：点击**下单结算**，生成订单（存档）。
3. `push`：快递**发货**，送到云端。

演示准备：

请进入 D:\Work\STM32\test 文件夹，新建一个文本文档 test.txt，在里面随便写一句 "Hello Git" 并保存关闭。（Git 只会追踪有内容的文件，空文件夹它是看不见的）。



第一步：添加到暂存区（`git add`）

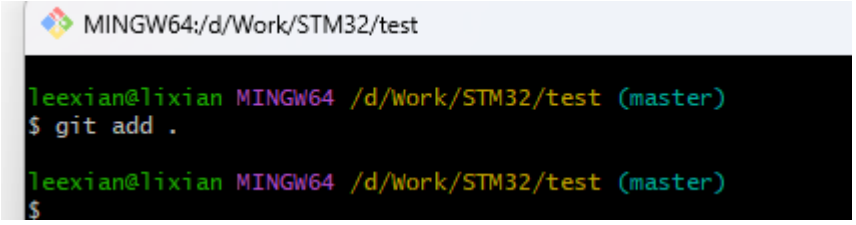
作用：告诉 Git，“我刚修改了文件，请把它们放进购物车，准备提交”。

- **操作**：在黑框中输入：（注意，需要你的黑框路径在你的库 D:\Work\STM32\test中，这点很重要，在文件夹的空白处 **右键** -> **Git Bash Here**）

  Bash

  ```
  git add .
  ```

- **细节详解**：
  - **空格**：注意 `add` 和 `.` 之间必须有一个空格。
  - **`.`（点）**：这是一个通配符，代表“当前目录下的**所有**变动”。
  - **效率**：不管你改了 1 个文件还是 100 个文件，用 `git add .` 可以一次性把它们全部装进购物车，最省事。



第二步：提交到本地仓库（`git commit`）

作用：将购物车里的内容**“下单结账”**，在本地生成一个永久的历史版本。

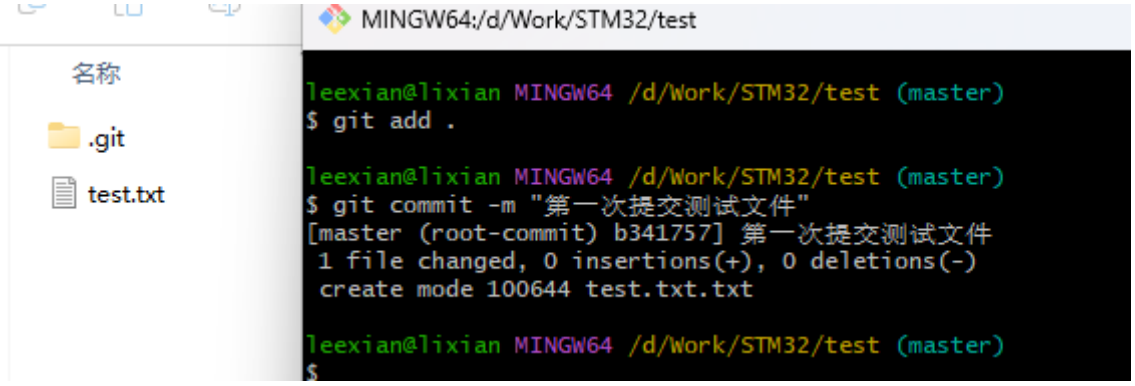
- **操作**：输入以下命令（注意引号）：

  Bash

  ```
  git commit -m "第一次提交测试文件"
  ```

- **细节详解**：
  - `-m`：是 `message`（信息）的缩写。
  - **双引号内容**：这是本次提交的**备注**。

- **重要性**：**备注要写人话！** 比如“修复电机BUG”或“新增PID参数”。千万别只写“update”或者“111”，否则一周后你自己都不知道这次改动是为了啥。



第三步：推送到 GitHub (`git push`)

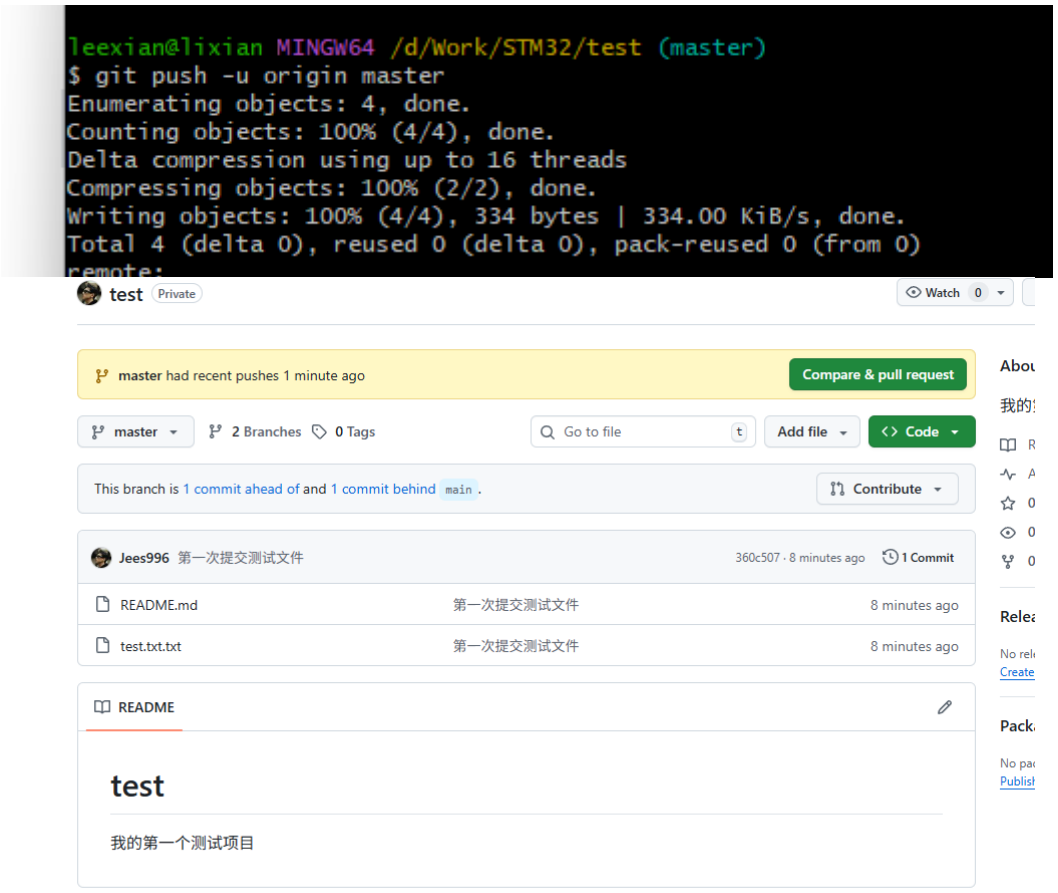
作用：将本地做好的存档**“同步”**到 GitHub 云端服务器。

- **操作**：输入：

  Bash

  ```
  git push
  ```

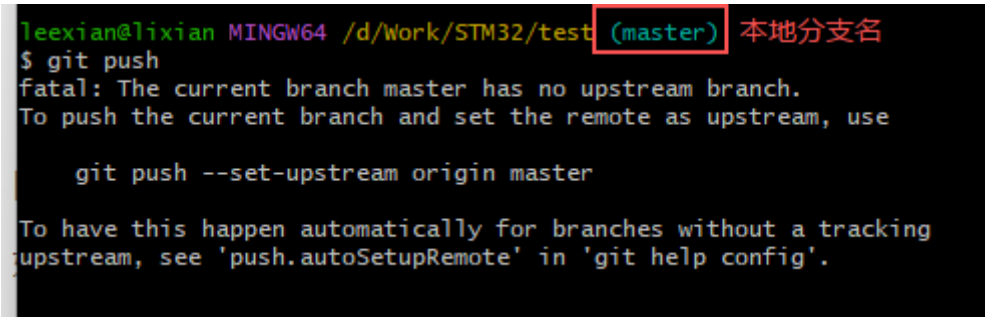
- **结果验证**：
  - 终端显示 `Writing objects: 100%` 即代表成功。
  - **此时去 GitHub 网页刷新一下你的仓库页面**，你会发现 `test.txt` 已经出现在那里了。



🚑 紧急救援：第一次 Push 报错了？

如果你在执行 `git push` 时，看到下面这行的报错：

```
fatal: The current branch master has no upstream branch.
```



别慌，这非常正常！

- **原因**：这是由于你本地创建仓库并初始化的时候没有使用GitHub提供的快捷命令，并且没有更改本地分支名，本地默认是master，云端默认是main，而导致推送失败。
- **解决办法**： 使用用下面这两行命令更改你本地分支名与云端对齐：

操作步骤（两行代码搞定）：

1. **本地改名**： 输入下面这行命令，把本地的 `master` 强制改名为 `main`：

   Bash

```
git branch -M main
```

- *现象：你会发现黑框框里的 `(master)` 瞬间变成了 `(main)`。*

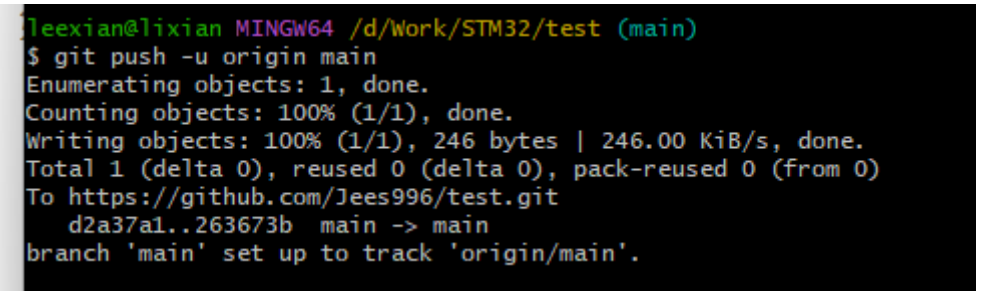
2. **推送并绑定**： 现在名字一样了，我们再次尝试推送，并告诉 Git “以后这就叫主分支”：

   Bash

   ```
   git push -u origin main
   ```

3. **成功标志**： 当看到 `Branch 'main' set up to track remote branch 'main' from 'origin'` 的提示时，说明**改名+上传+绑定**全部搞定！ 以后再修改代码，直接输简单的 `git push` 就可以了。

以后还用输这么长吗？ 不用！加了这个 `-u` 之后，Git 就记住这条路了。**以后你只需要输 `git push` 这两个词，它就能自动识别路径了。**



2、查看历史记录(`git log`)

代码写着写着忘了几号版本是能跑的？或者想揪出是谁写了那个 Bug？这时候就需要查查“家谱”了。

操作步骤：

1. **输入命令**： 在终端里输入：

   Bash

   ```
   git log
   ```

   (强烈推荐加 `--oneline` 这个小尾巴，它会让日志变得非常简洁，一行一条，不占地儿。`git log --oneline`)

2. **看懂信息**： 屏幕上会吐出一串列表，你只需要关注两个核心信息：
   - **黄色的乱码（Commit ID）**：例如 `a1b2c3d`。
     - **这是啥？** 它是版本的身份证号，也是你穿越时空的**“机票代码”**。想回到这个版本，全靠这串数。
   - **后面的文字**：这就是你 `commit -m` 时写的备注。现在知道为什么要好好写备注了吧？

3. 🚨 **逃生指南（按 Q 退出）** 如果你的历史记录太长，屏幕最下面出现了冒号 `:`，这时候你会发现光标卡住了。
    - **动作**：按一下键盘上的字母 `q` (Quit)，就能瞬间退出查看模式，回到黑框框。

3 版本回退 (`git reset`)

当你确定现在的代码已经烂到没救了，想彻底回到昨天那个“能跑”的版本。Git 能让你一键穿越。

操作步骤：

1. **找票根（版本号）**： 先用上面的 `git log --oneline` 找到你想穿越回去的那个版本的 **Commit ID**。
    - *不需要复制全部，复制前 6-7 位就够了，比如 `e3d2a1`。*
2. **启动时光机**： 输入下面这行命令（注意空格，把下面的 `e3d2a1` 换成你查到的号码）：

    Bash

    ```
    git reset --hard e3d2a1
    ```

3. **见证奇迹**： 按回车。Git 会提示 `HEAD is now at...`。 这时候你去打开你的代码文件夹，你会发现**所有的文件都瞬间变回了那一刻的样子**！

⚠️ 安全须知

关于 `--hard` 参数： 命令里那个 `--hard`（硬复位）是个狠角色。

- **它的意思是：“别废话，把我现在所有改乱的东西全扔掉，强制恢复到那个版本。”**
- **副作用：你从“那个版本”之后写的所有新代码，都会瞬间灰飞烟灭，找不回来的那种！**
- **建议**：在使用这招之前，确保你现在手头没有重要的新代码，或者先把重要的文件另外备份一份。

4、检查当前状态 (`git status`)

这是新手最容易忽略，但老手用得最多的指令。当你不知道自己改了哪些文件，或者不知道刚才 `add` 成功没有，输它就对了。建议养成**“没事就输一下”**的好习惯。

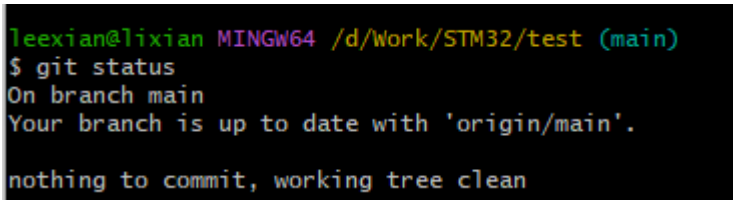
操作步骤：

1. **输入命令**：

   Bash

   ```
   git status
   ```

2. **看懂颜色（红绿灯）**：
   - 🔴 **红色的字**：代表文件**被修改了**，但还没进购物车（还没 `add`）。
   - 🟢 **绿色的字**：代表文件**已经在购物车里了**（已经 `add`），随时可以下单（`commit`）。
   - **nothing to commit, working tree clean**：代表此时此刻，你的本地代码非常干净，和仓库里的一模一样，没有任何变动。



5、拉取最新代码（`git pull`）

在战队里合作开发，最常听到的就是队友吼一嗓子："代码我推上去了，你拉一下！"这时候千万别去网页下载压缩包，太 Low 了。

场景： 队友更新了视觉识别代码并上传了 GitHub，你需要把他的更新同步到你的电脑上。

操作步骤：

1. **输入命令**：

   Bash

   ```
   git pull
   ```

2. **结果说明**：
   - **Fast-forward / Updating...**：屏幕滚动代码，最后显示 `100%`。说明队友的代码已经自动下载并合并进你的文件夹里了，直接打开用就行。
   - **Already up to date**：说明你是最新的，队友在骗你（或者他忘了 Push）。

6、给分支改名 (git branch -M)

场景：嫌名字不好听，或者为了解决"本地 master 与云端 main 不一致"的问题。

操作步骤： 直接在黑框里输入：

Bash

```
git branch -M main
```

(注：如果你想改成其他名字，把 main 换成你想要的英文名即可)

结果验证： 看光标前面的括号，会瞬间从 `(旧名字)` 变身 `(新名字)`。

后续操作： 改名后，记得重新绑定云端，否则 Git 找不到路：

Bash

```
git push -u origin main
```

待补充更新！

5、进阶高级指令（多人协作）

待补充更新！

常用指令速查表

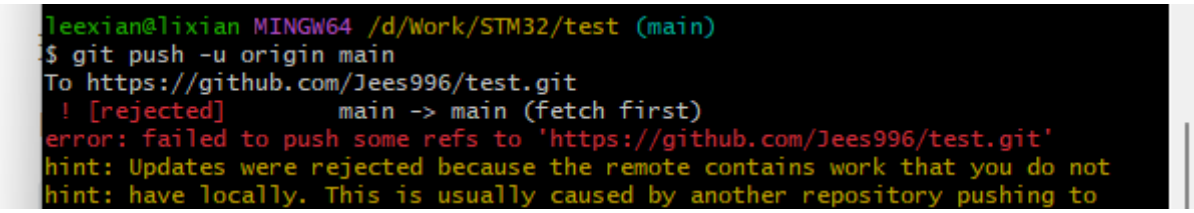
指令	核心功能	说明与备注
`git init`	**初始化仓库**	在当前目录下生成 `.git` 目录，使其成为受 Git 管理的本地仓库。
`git status`	**查看仓库状态**	检查工作区文件的变动状态（如已修改、未暂存、未追踪），建议高频使用。
`git add .`	**添加所有文件**	将当前目录下的所有文件变更（增、删、改）添加到暂存区。
`git commit -m "..."`	**提交到本地**	将暂存区的内容正式归档，生成一个新的版本记录。引号内必须填写具体修改备注。
`git log --oneline`	**查看提交历史**	以单行简洁格式显示历史提交记录与 Commit ID（版本号）。

指令	核心功能	说明与备注
`git branch -M main`	**重命名分支**	将当前分支强制重命名为 `main`，常用于对齐 GitHub 默认主分支名称。
`git remote add origin ...`	**绑定远程地址**	建立本地仓库与远程 GitHub 仓库的关联，并将远程地址命名为 `origin`。
`git push`	**推送至远程**	将本地仓库的提交记录同步上传到 GitHub 远程仓库。（首次需加 `-u origin main`）
`git pull`	**拉取远程代码**	从 GitHub 下载最新的代码并自动合并到本地当前分支。
`git reset --hard [ID]`	**版本回退**	强制将工作区回退到指定版本。**警告：该操作不可逆，会丢弃所有未提交的修改。**

常见问题解决办法

这里收集了新手最容易遇到的“红字报错”和“灵异现象”。如果你卡住了，先来这里找找解药。

1. 推送报错，云端与本地文件不同步，`! [rejected] ... (fetch first)`



现象描述： 当你兴冲冲地执行 `git push` 时，终端给你泼了一盆冷水，出现类似下面的红字：

Bash

```
! [rejected]        main -> main (fetch first)
error: failed to push some refs to '...'
hint: Updates were rejected because the remote contains work that you do not have locally...
```

人话翻译（原因）： **“撞车了！”** 这就好比你和队友同时改一个文档。GitHub 发现它的服务器上有一个文件（通常是你创建仓库时手滑勾选生成的 `README.md`），但是你电脑本地的文件夹里**没有**这个文件。GitHub 觉得：“咦？你这历史记录对不上啊？要是让你强行上传，我这儿的文件就被你覆盖丢了！”于是它无情地拒绝了你。

解决办法（三步走）：

我们要做的就是：**先把天上的文件“拉”下来，强行和本地的合并，然后再一起“推”上去。**

第一步：强行合并（拉取） 在黑框框里输入下面这行“万能灵药”命令：

Bash

```
git pull origin main --allow-unrelated-histories
```

- *解释：`--allow-unrelated-histories` 就是告诉 Git："别管它俩以前是不是一家人，现在强行把它们撮合在一起。"*

第二步：逃离"黑/蓝屏"编辑器（Vim 逃生指南） 输入上面的命令回车后，你的黑框框很可能会瞬间变样，出现满屏的波浪线 `~` 或者黄色的文字。 别慌！这是 Git 自动打开了 Vim 编辑器让你写备注。我们不需要写，只需要**安全退出**：

1. 按一下键盘左上角的 `Esc` 键（确保退出了编辑模式）。
2. 输入冒号和字母 `:wq` （注意是英文冒号，w代表保存，q代表退出）。
    - *屏幕左下角会出现 `:wq` 字样。*
3. 按 **回车 (Enter)**。
    - *此时你会回到熟悉的命令行界面。*

第三步：再次推送 现在本地和云端已经统一了，再次尝试上传：

Bash

```
git push -u origin main
```

这次你应该就能看到舒服的 `Writing objects: 100%` 成功提示了！

leexian@lixian MINGW64 /d/Work/技术文档/easy-git-cn (main)
$ git push
fatal: unable to access 'https://github.com/BJFU-Forestry-Wolves/easy-git-cn.git/': Recv failure: Connection was reset